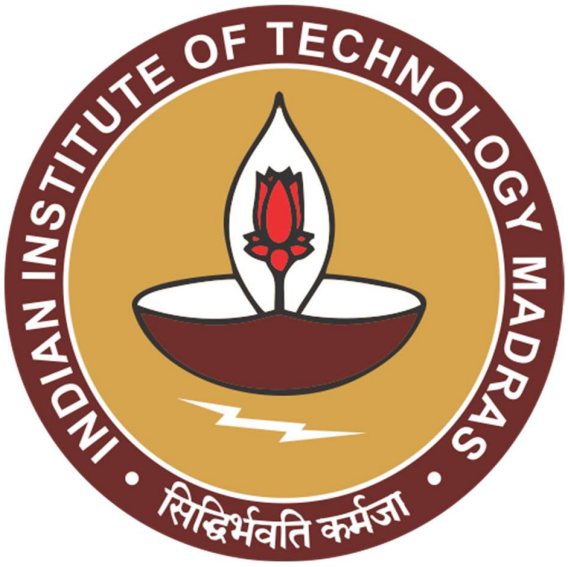


Modern App Development – II

Topic:

I-School Assessment Platform

Submitted By:

Daiwik Rankawat

Github Repo:

https://github.com/daiwik-project/MAD-2-Project-iitm/

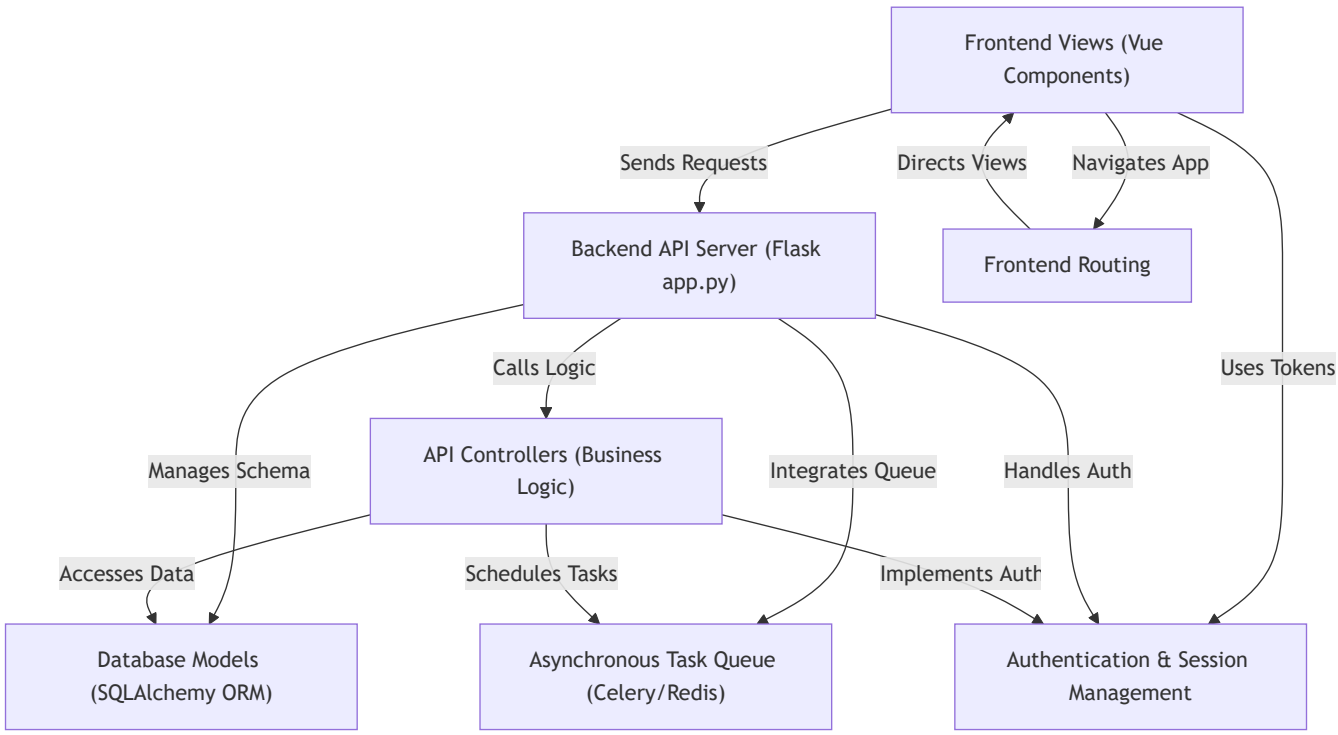
Test URL:

https://courageous-faloodeh-5b90a2.netlify.app/

Tutorial: MAD-2-Project-iitm

This project is an **online examination and learning platform** that allows users to *take quizzes*, *track their performance*, and *manage their learning preferences*. It also provides a comprehensive **admin panel** for educators to *create and manage content* like levels, subjects, chapters, quizzes, and questions, as well as *control user access* and *monitor overall platform activity*. Background tasks ensure *timely notifications* and *performance reports*.

Visual Overview



Chapters

1. Database Models (SQLAlchemy ORM)
2. Frontend Views (Vue Components)
3. Frontend Routing
4. Authentication & Session Management
5. Backend API Server (Flask `app.py`)
6. API Controllers (Business Logic)
7. Asynchronous Task Queue (Celery/Redis)

Chapter 1: Database Models (SQLAlchemy ORM)

Welcome to the first chapter of our tutorial for the `MAD-2-Project-iitm`! We're starting at the very foundation: how our application understands and stores all the important information it uses.

What's the Big Idea? (Motivation)

Imagine you're building a super organized library. You have different types of "books" – maybe user accounts, quizzes, or learning chapters. How do you keep track of all the details for each "book" in a consistent way?

If you just started writing down notes on scraps of paper, things would quickly get messy. You'd need a clear system! This is exactly the problem **Database Models** solve for our application.

Instead of writing complex instructions directly for a database (which is like talking in a very specific, technical language called SQL), we use **Python objects** as **blueprints**. These blueprints are called **Database Models**.

Think of a Database Model as a **detailed form** or a **template**. For instance, we'll have a "User Form" blueprint that says: "Every user needs a username, an email, a password, and a unique ID." When a new user signs up, we just fill out a new "User Form" based on this blueprint.

This makes it much easier for our Python application to store, find, and change information without needing to speak complex database "languages."

Key Concepts Explained

Before we dive into the code, let's understand some basic terms:

- **Database**: This is like a giant, super-organized digital filing cabinet where we store all our application's information.
- **Table**: Inside a database, information is organized into tables. Each table holds data of a specific type. For example, we might have a `Users` table, a `Quizzes` table, or a `Chapters` table.
    - *Analogy*: If the database is a filing cabinet, a table is a specific drawer labeled "Users" or "Quizzes."
- **Row (or Record)**: Each individual item in a table is called a row. So, one specific user's details would be one row in the `Users` table.
    - *Analogy*: If a table is a drawer, a row is a single file folder inside that drawer, containing all the info for one specific user.
- **Column (or Field)**: Each piece of information within a row is a column. For a user, columns might be `username`, `email`, `password`, etc.
    - *Analogy*: Inside that file folder, columns are the specific headings on a form, like "Name:", "Email:", "Password:".

Now, let's talk about the magic behind our models:

- **SQLAlchemy ORM**: ORM stands for **Object-Relational Mapping**. SQLAlchemy is a powerful tool in Python that acts as a "translator." It lets us define our database tables (our "forms") using regular Python code (our "objects") instead of complex database commands. It then handles all the translation behind the scenes.

    - *Analogy*: SQLAlchemy is like a universal translator that lets you speak Python to the database, and it converts your Python requests into the database's language (SQL).

- **Database Model (Our Focus!)**: In our project, a Database Model is a Python class (a blueprint) that defines the structure of a table in our database. It tells SQLAlchemy: "Here's what a `User` looks like in the database, and here are all the pieces of information (columns) it should contain."

Setting Up Our Database Connection

First, our application needs a way to talk to the database. We set this up in a file dedicated to our database connection:

```
# File: backend/database.py
from flask_sqlalchemy import SQLAlchemy

db = SQLAlchemy()
```

In this small file:

- We import `SQLAlchemy`, which is the main tool.
- `db = SQLAlchemy()` creates a special `db` object. This `db` object is what we'll use throughout our application to interact with the database using SQLAlchemy. Think of `db` as our main gateway to all database operations.

Defining Our Database Models

Now, let's look at how we define our blueprints (models) in our project. You'll find these in `backend/models/model.py`.

Example: The `User` Model

Let's pick out the `User` model as our primary example. This model is the blueprint for storing information about every user in our system:

```
# File: backend/models/model.py (Excerpt)
from datetime import datetime
from database import db # Our special db object

class User(db.Model): # This class is our blueprint for User data
    uuid = db.Column(db.String(12), unique=True, nullable=False, primary_key=True)
    username = db.Column(db.String(80),  nullable=False)
    email = db.Column(db.String(120), nullable=False)
    password = db.Column(db.String(120), nullable=False)
    is_active = db.Column(db.Boolean, default=True)
    created_at = db.Column(db.DateTime, default=datetime.utcnow)
    # ... other columns omitted for brevity
```

Let's break down what's happening here:

- `class User(db.Model):`: This line tells SQLAlchemy that our `User` class is a **database model**. It inherits special powers from `db.Model`, allowing it to represent a table in our database.
- `uuid = db.Column(...)`: Each line inside the `User` class (like `uuid`, `username`, `email`) defines a **column** in our database table.
- `db.Column()`: This is how we create a column. Inside the parentheses, we define its characteristics:
    - `db.String(12)`: This means the column will store text (a string), and `(12)` means it can hold up to 12 characters. Other types include `db.Integer` (whole numbers), `db.Boolean` (True/False), `db.DateTime` (dates and times), and `db.Text` (longer text).
    - `unique=True`: This ensures that no two users can have the same `uuid`. It must be unique across all users.
    - `nullable=False`: This means this column *must* have a value; it cannot be empty.
    - `primary_key=True`: This makes `uuid` the unique identifier for each row in the `User` table. It's how we specifically identify one user from another.
    - `default=...`: Sets a default value if one isn't provided (e.g., `created_at` automatically gets the current time).

In short, this `User` class is our Python-friendly way of saying: "Dear database, please create a table named `user` (SQLAlchemy usually makes table names lowercase and plural) with columns for `uuid`, `username`, `email`, `password`, etc., and set them up with these specific rules."

Connecting Models with Relationships (Foreign Keys)

Real-world data is rarely isolated. Users are associated with quizzes, quizzes belong to chapters, and chapters belong to subjects. Database models handle these connections using **relationships** and **foreign keys**.

Let's look at `Subject` and `Chapter` models:

```
# File: backend/models/model.py (Excerpt)
# ...
class Subject(db.Model):
    uuid = db.Column(db.String(12), unique=True, nullable=False, primary_key=True)
    level_uuid = db.Column(db.String(12), db.ForeignKey('level.uuid'),
nullable=False)
    name = db.Column(db.String(100), nullable=False)
    # ...
    chapters = db.relationship('Chapter', backref='subject') # Link to Chapter
model

class Chapter(db.Model):
    uuid = db.Column(db.String(12), unique=True, nullable=False, primary_key=True)
    name = db.Column(db.String(100), nullable=False)
    subject_uuid = db.Column(db.String(12), db.ForeignKey('subject.uuid'),
nullable=False)
    # ...
```

- `subject_uuid = db.Column(db.String(12), db.ForeignKey('subject.uuid'), nullable=False)`:
    - This line in the `Chapter` model creates a `subject_uuid` column.

    - `db.ForeignKey('subject.uuid')` is the crucial part! It tells the database that the value in this column *must* match a `uuid` from the `Subject` table.
    - *Analogy*: Imagine a "Chapter" form. This `subject_uuid` field says, "This chapter belongs to a specific subject, and I'll write down that subject's unique ID here." This creates a link, so we know which subject each chapter is part of.

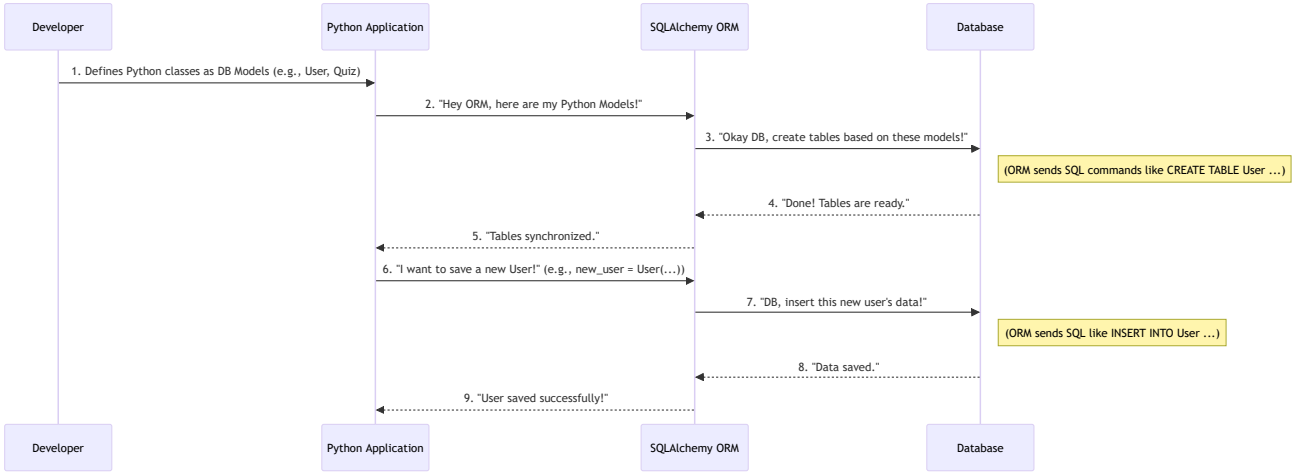
- `chapters = db.relationship('Chapter', backref='subject')`:
    - This line in the `Subject` model doesn't create a column in the database table itself. Instead, it's a special SQLAlchemy helper.
    - It tells SQLAlchemy: "A `Subject` can be related to many `Chapters`."
    - `backref='subject'` means that from any `Chapter` object, you can easily access its related `Subject` object (e.g., `my_chapter.subject`). This makes navigating related data very convenient in our Python code.

These relationships are fundamental because they model how different pieces of information in our application are connected.

What Happens Under the Hood?

You might be wondering: How does writing Python code in `model.py` actually create tables in a database? This is where SQLAlchemy's "translation" power comes in.

Here's a simplified sequence of events:



1. **You Define Models**: As a developer, you write the Python classes (`User`, `Quiz`, `Chapter`, etc.) in `backend/models/model.py`, inheriting from `db.Model` and defining columns.
2. **SQLAlchemy Reads Blueprints**: When our Python application starts up, SQLAlchemy reads these Python classes. It understands that `User` should become a `user` table, `username` should be a text column, `uuid` a primary key, and so on.
3. **Translates to SQL**: SQLAlchemy then translates these Python definitions into actual database commands (like `CREATE TABLE user (uuid VARCHAR(12) PRIMARY KEY, username VARCHAR(80), ...)`).
4. **Database Creation**: These SQL commands are sent to our database (which could be something like SQLite, PostgreSQL, or MySQL). The database then creates the actual tables, columns, and sets up all the rules (like `unique` or `nullable`) exactly as defined in your Python models.

5. **Interaction**: Later, when your application wants to save a new user, it creates a Python `User` object, fills its details, and tells SQLAlchemy to save it. SQLAlchemy again translates this into a database command (like `INSERT INTO user (...) VALUES (...)`) and sends it to the database.

This entire process means you spend most of your time working with familiar Python objects, and SQLAlchemy handles the intricate communication with the database for you!

Summary

In this chapter, we learned that:

- **Database Models** are Python blueprints for storing different types of information (like Users, Quizzes, Chapters) in our database.
- We use **SQLAlchemy ORM** as a powerful "translator" that lets us define these blueprints and interact with the database using Python objects, avoiding complex SQL commands directly.
- Models are defined as Python classes inheriting from `db.Model`, with `db.Column` defining each piece of information (column) and its rules.
- **Foreign Keys** and `db.relationship` help us link different models together, reflecting how data connects in the real world (e.g., a Chapter belongs to a Subject).

Understanding these database models is crucial because they define the very structure of all the data our application will manage. In the next chapter, we'll shift our focus to the visible part of our application – how we display and interact with this data using **Frontend Views (Vue Components)**.

Next Chapter: Frontend Views (Vue Components)

Chapter 2: Frontend Views (Vue Components)

Welcome back! In our first chapter , we explored how our application neatly stores information using **Database Models**. We learned that these models are like "blueprints" for the data, ensuring everything is organized in our digital filing cabinet (the database).

But what's the point of having a super-organized library if no one can actually *read* the books or interact with them? This is where **Frontend Views** come in!

What's the Big Idea? (Motivation)

Imagine our application is a house. The Database Models (SQLAlchemy ORM) define the foundation, the structure of each room, and what kind of items (data) can be stored in each. Now, we need to actually *build* the rooms that people can walk into, see, and use!

This is the job of **Frontend Views**. They are the **visible parts** of our website that users interact with.

For example, when you open the `MAD-2-Project-iitm` application, you'll see a beautiful **Dashboard page**. This page shows you a list of "Chapters" and "Upcoming Quizzes." You can click on them, take quizzes, or view your profile. All these interactive elements and visible sections are built using Frontend Views.

Without Frontend Views, our application would just be a bunch of data sitting in a database, completely hidden from the user. Frontend Views bring our application to life, allowing users to **see**, **understand**, and **interact** with the data we meticulously structured in the backend.

In our project, we use a popular tool called **Vue.js** to build these views. Each distinct screen or interactive element you see on the website, like the Dashboard, a Quiz page, or a Login form, is built as a **Vue Component**.

Think of Vue Components as **individual, specialized rooms in our application's house**. Each room (component) has its own design, purpose, and logic, all working together to create a smooth and rich user experience.

Key Concepts Explained

Let's break down the core ideas behind Frontend Views and Vue Components.

1. What is a Frontend View?

A **Frontend View** is simply what the user *sees* and *interacts with* in their web browser. It's the user interface (UI) of our application. It takes the data from our Backend API Server (Flask `app.py`) and presents it in a human-friendly way.

2. What is a Vue Component?

In Vue.js, we build our Frontend Views using **Components**. Imagine building with LEGO bricks. Each LEGO brick is a component:

- It has a specific shape and color (its **visual appearance**).
- It can do something (its **behavior**).

- You can connect it to other bricks (it can **interact with other components**).

A Vue Component is a self-contained, reusable block of user interface code. Instead of writing one giant HTML file for the whole website, we break it down into smaller, manageable components.

For example, a "Dashboard" might be one big component, but inside it, there could be smaller "Chapter Card" components, "Quiz List" components, and a "Navigation Bar" component. This makes building and maintaining complex interfaces much easier!

3. The Single File Component (`.vue` file)

In our project, each Vue Component lives in a special file that ends with `.vue`. These are called **Single File Components (SFCs)**. They cleverly combine three things in one place:

Section	Purpose	Analogy
`<template>`	This is where you write the **HTML structure** of your component. It defines *what* the component looks like on the screen.	The **walls, windows, and furniture** layout of a room.
`<script>`	This is where you write the **JavaScript logic** for your component. It defines *how* the component behaves, what data it needs, and what actions it can perform.	The **electrical wiring, plumbing, and smart home system** that make the room functional.
`<style scoped>`	This is where you write the **CSS styling** for your component. It defines *how* the HTML elements look (colors, fonts, layout). The `scoped` keyword ensures these styles only affect *this specific component*, preventing them from accidentally changing other parts of your website.	The **paint color, wallpaper, and decor** that give the room its unique look.

4. Component Data and Methods

Inside the `<script>` section of a Vue Component, two very important parts are `data()` and `methods`:

- `data()`: This is a function that returns an object containing all the **information (variables)** that this specific component needs to keep track of.
    - *Example*: A `Dashboard` component might have `data` for `chapters` (a list of all available chapters) and `quizzes` (a list of upcoming quizzes).
    - Think of it as the component's **memory** or its **current state**.
- `methods`: This is an object containing all the **actions (functions)** that this component can perform.
    - *Example*: A `Dashboard` component might have methods like `fetchChapters()` (to get chapters from the backend) or `startQuiz()` (when a user clicks to begin a quiz).
    - Think of it as the component's **actions** or **abilities**.

Bringing it to Life: The Dashboard Example

Let's look at a simplified version of `frontend/src/views/user/dashboard.vue`. This component is responsible for displaying the user's main dashboard.

1. The `<template>` Section (What it Looks Like)

The template defines the visual structure. Notice how it uses special Vue features like `v-for` to repeat elements and `{{ }}` to display data.

```
<!-- File: frontend/src/views/user/dashboard.vue (Simplified Excerpt) -->
<template>
    <div class="dashboard-page-wrapper">
        <!-- Navbar (Navigation at the top) -->
        <nav class="navbar">
            <h1>My Dashboard</h1>
            <!-- ... other navigation links ... -->
        </nav>

        <div class="dashboard-container">
            <section class="chapters-section">
                <h2>Your Chapters</h2>
                <div class="chapters-grid">
                    <!-- This 'v-for' loops through each 'chapter' in the
'chapters' list -->
                    <div class="chapter-box" v-for="chapter in chapters"
:key="chapter[0]">
                        <h3 class="chapter-box-title">{{ chapter[1] }}</h3>
                        <p class="chapter-box-description">{{ chapter[2] }}</p>
                        <!-- 'chapter[0]' is typically the unique ID, 'chapter[1]'
the title, 'chapter[2]' the description -->
                        <button class="btn btn-view-chapter"
@click="view_the_chap(chapter[0])">
                            View Chapter
                        </button>
                    </div>
                </div>
            </section>

            <!-- Section for Quizzes (Similar structure, omitted for brevity) -->
            <section class="quizzes-section">
                <h2>Upcoming Quizzes</h2>
                <!-- ... table of quizzes using v-for ... -->
            </section>
        </div>
    </div>
</template>
```

Explanation:

- `div` and `section`: These are standard HTML tags that organize the content.
- `v-for="chapter in chapters"`: This is a special Vue **directive**. It tells Vue to take the `chapters` list (which comes from our component's `data`) and create a new `chapter-box div` for *each item* in that

list. This is super powerful for displaying lists of items!

- `:key="chapter[0]"`: Vue needs a unique `key` for each item in a `v-for` list to keep track of them efficiently. We use the chapter's unique ID (`chapter[0]`).
- `{{ chapter[1] }}` and `{{ chapter[2] }}`: These are Vue's **interpolation** syntax. They tell Vue to take the values of `chapter[1]` (the chapter's name/title) and `chapter[2]` (its description) from the current `chapter` item and display them as plain text.
- `@click="view_the_chap(chapter[0])"`: This is another Vue **directive**. It listens for a "click" event on the button. When clicked, it calls the `view_the_chap` method (defined in our `<script>` section) and passes the `chapter[0]` (the chapter's ID) as an argument.

2. The `<script>` Section (How it Behaves)

The script section manages the component's data and defines its actions.

```
// File: frontend/src/views/user/dashboard.vue (Simplified Excerpt)
<script>
import axios from 'axios'; // Tool to make network requests (talk to backend)
import vuecookies from 'vue-cookies'; // Tool to manage web cookies (for
authentication)

export default {
    name: 'DashboardPage', // Name of our component
    data() { // This function returns the data for this component
        return {
            chapters: [], // An empty list, will be filled with data from backend
            quizzes: [],  // Another empty list for quizzes
        };
    },
    methods: { // This object holds all the functions (actions) for this component
        async list_all_chapters_n_quiz() {
            try {
                // Get the user's authentication token from cookies
                const token = vuecookies.get('access_token');

                // Make a request to our backend API to get chapters and quizzes
                // The URL '/dashboard/chapter_n_quiz' points to our Flask backend
                const response = await
axios.get(`http://127.0.0.1:5000/dashboard/chapter_n_quiz?token=${token}`);

                // Update the component's 'chapters' and 'quizzes' data
                // This automatically updates the <template> part thanks to Vue's
magic!
                this.chapters = response.data.chapters;
                this.quizzes = response.data.quiz;
            } catch (error) {
                console.error('Error fetching data:', error);
            }
        },
        view_the_chap(chapter_id){
            // When a user clicks "View Chapter", navigate to a new page
            // This will be covered in detail in the [Frontend Routing]
```

```
(03_frontend_routing_.md) chapter
            this.$router.push(`/dashboard/chapter/${chapter_id}`);
        },
        // ... other methods like startQuiz, reattemptQuiz (omitted for brevity)
    },
    mounted() { // This is a special "lifecycle hook" in Vue
        // 'mounted' means: "After the component has been added to the page, do
this!"
        this.list_all_chapters_n_quiz(); // Call our method to fetch data as soon
as the page loads
    }
}
</script>
```

Explanation:

- `import axios from 'axios';`: `axios` is a popular library for making HTTP requests (like fetching data) to a server. We'll learn more about the server in Backend API Server (Flask `app.py`).
- `import vuecookies from 'vue-cookies';`: This library helps us read and write small pieces of information (like user login tokens) that browsers store, called "cookies." This will be vital for Authentication & Session Management.
- `data() { return { chapters: [], quizzes: [] }; }`: This sets up initial empty lists for `chapters` and `quizzes`. When the data is fetched, these lists will be populated.
- `async list_all_chapters_n_quiz()`: This method is `async` because it needs to wait for the `axios.get` request to the backend to complete before it can use the `response`.
  - `axios.get(...)`: This sends a "GET" request to our backend server (e.g., `http://127.0.0.1:5000/dashboard/chapter_n_quiz`).
  - `this.chapters = response.data.chapters;`: Once the backend sends the data, we update `this.chapters` and `this.quizzes` with the new information. Vue automatically notices this change and updates the HTML in the `<template>` section, thanks to its "reactivity" system.
- `mounted()`: This is a **lifecycle hook**. Vue components go through different stages (like being created, added to the page, updated, and removed). `mounted` runs once, right after the component first appears on the screen. It's the perfect place to fetch initial data.
- `view_the_chap(chapter_id)`: This method uses `this.$router.push` to change the URL in the browser and load a different component. We'll cover this in Frontend Routing.

3. The `<style scoped>` Section (How it Looks)

This section contains CSS rules that make our component look pretty. The `scoped` attribute ensures these styles don't accidentally affect other parts of our website.

```
/* File: frontend/src/views/user/dashboard.vue (Simplified Excerpt) */
<style scoped>
.dashboard-page-wrapper {
    background-color: #f0f2f5;
    min-height: 100vh;
}

.dashboard-container {
```

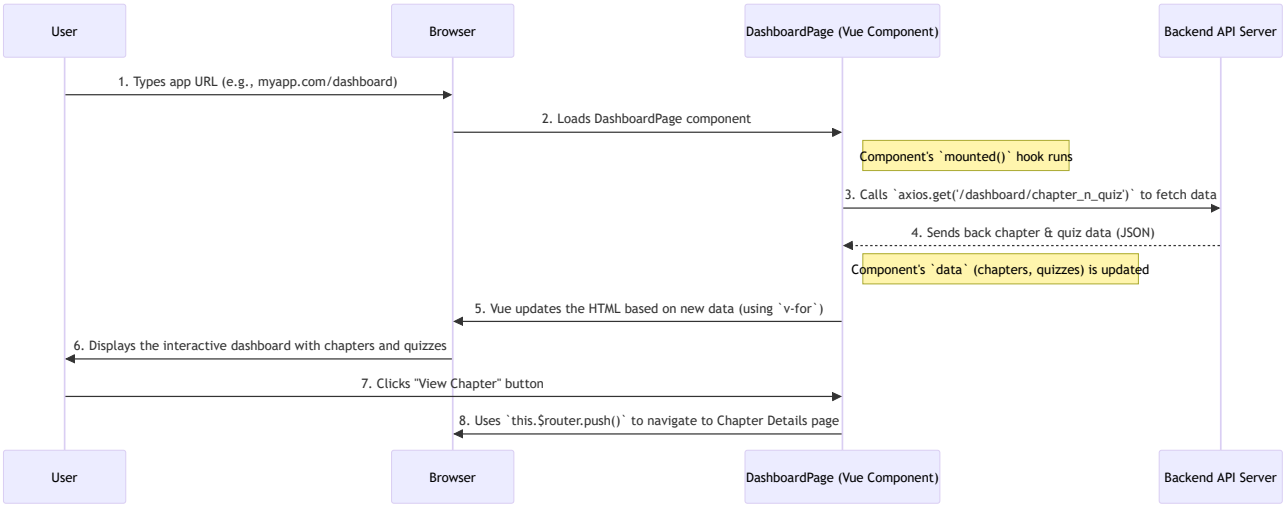
```
    background-color: #ffffff;
    max-width: 1400px;
    margin: 30px auto;
    padding: 30px 40px;
    border-radius: 12px;
    box-shadow: 0 8px 25px rgba(0, 0, 0, 0.1);
}

.chapter-box {
    background: #ffffff;
    border: 1px solid #e0e6ed;
    border-radius: 10px;
    padding: 25px;
    text-align: center;
    box-shadow: 0 4px 15px rgba(0, 0, 0, 0.05);
    transition: transform 0.3s ease;
}

.chapter-box:hover {
    transform: translateY(-8px) scale(1.02);
    box-shadow: 0 10px 25px rgba(52, 152, 219, 0.2);
}

.btn-view-chapter {
    background: linear-gradient(135deg, #3498db, #2980b9);
    color: white;
    padding: 10px 20px;
    border-radius: 20px;
}

/* ... more CSS styles ... */
</style>
```

Explanation:

- These are standard CSS rules. For example, `.dashboard-container` styles the main white box of the dashboard, giving it a background color, max-width, padding, rounded corners (`border-radius`), and a shadow.
- `.chapter-box:hover` uses a CSS `transition` and `transform` to create a smooth lifting effect when you move your mouse over a chapter box, making it more interactive.
- The `scoped` attribute means `.chapter-box` styles will *only* apply to `div` elements with the class `chapter-box` *within this specific* `DashboardPage` *component*, ensuring our styles don't clash with similarly named classes in other components.

What Happens Under the Hood?

Let's visualize the journey of a Frontend View (Vue Component) from the user opening the app to seeing the data.



1. **User Accesses URL**: The user types the application's URL in their browser (e.g., `myapp.com/dashboard`).
2. **Browser Loads Component**: The browser loads the main Vue application, which then figures out that the `/dashboard` URL should show the `DashboardPage` Vue Component.
3. **Component Initializes**: The `DashboardPage` component starts up. Its `mounted()` lifecycle hook runs immediately.
4. **Component Fetches Data**: Inside `mounted()`, our `list_all_chapters_n_quiz()` method is called. This method uses `axios` to send a request to our Backend API Server (Flask `app.py`) to get the latest chapters and quizzes.
5. **Backend Responds**: The Backend API processes the request (potentially using API Controllers (Business Logic) to interact with Database Models (SQLAlchemy ORM)) and sends back the chapter and quiz data, usually in a format called JSON.
6. **Component Updates UI**: The Vue Component receives this data and updates its `chapters` and `quizzes` variables. Because Vue is "reactive," any changes to these `data` variables automatically trigger updates in the `<template>` part of the component. For example, the `v-for` loop redraws the chapter boxes with the newly fetched information.
7. **User Interacts**: The user sees the beautiful, updated dashboard. If they click a button (like "View Chapter"), the `@click` directive triggers a method in the component.
8. **Component Navigates**: That method, for example `view_the_chap`, uses Vue's routing system (which we'll discuss in the next chapter) to change the URL and load a new Frontend View.

This entire dance between the Frontend View and the Backend API is what makes modern web applications dynamic and interactive, providing a rich user experience!

Summary

In this chapter, we learned that:

- **Frontend Views** are the interactive, visible parts of our application that users see and use.
- We use **Vue Components** (files ending in `.vue`) to build these views. Each component is a self-contained unit with:
  - A `<template>` for its HTML structure.
  - A `<script>` for its JavaScript logic (data, methods).
  - A `<style scoped>` for its CSS styling.
- Vue's directives like `v-for` help us display lists, and `@click` helps us handle user interactions.

- Components fetch dynamic data from our backend using tools like `axios` and update the UI automatically.

Understanding Frontend Views is key to building a user-friendly application. In the next chapter, we'll learn how users can seamlessly move between these different views (or "rooms") using **Frontend Routing**.

Next Chapter: Frontend Routing

Chapter 3: Frontend Routing

Welcome back! In our previous chapter, we learned how to build individual "rooms" or screens for our application using **Frontend Views (Vue Components)**. We saw how a `DashboardPage` component could display chapters and quizzes, and how a `QuizAttempt` component could handle a quiz session.

But what if you want to go from the Dashboard to a specific Chapter's details page, or from a quiz summary to your user profile? How do you move between these different "rooms" in our application without closing the whole house and reopening it?

This is where **Frontend Routing** comes in!

What's the Big Idea? (Motivation)

Imagine our application is a large building with many specialized rooms. The Frontend Views (Vue Components) are these individual rooms – a "Lobby" (Login page), a "Study Hall" (Dashboard), a "Lecture Room" (Chapter page), and a "Testing Center" (Quiz page).

Without a clear system, moving between these rooms would be chaotic. You'd have to exit the entire building and re-enter a different door each time you wanted to go to a new room. This is similar to how old websites used to work: clicking a link would cause the *entire page* to reload, which feels slow and clunky.

Frontend Routing is like the building's **internal GPS system and directory**. It tells your browser:

- "If the address is `/dashboard`, show the Dashboard room."
- "If the address is `/dashboard/chapter/123`, show the Chapter Details room for chapter number 123."

This system allows for **smooth, instant transitions** between different parts of our website without the entire page having to reload each time. It makes our web application feel fast and seamless, just like a desktop application.

Our goal in this chapter is to understand how we set up this "internal GPS" so that when a user clicks "View Chapter" on the dashboard, they instantly land on the correct chapter's details page, just like we hinted at with `this.$router.push` in the last chapter.

Key Concepts Explained

Let's break down the core ideas behind Frontend Routing in our Vue.js application.

1. What is a "Route"?

In the context of frontend routing, a **Route** is simply a **mapping** between a specific **URL path** (what you see in your browser's address bar) and a **Vue Component** (the "room" we want to display).

For example:

- The path `/dashboard` maps to our `DashboardPage` component.
- The path `/login` maps to our `Login` component.

2. Vue Router: Our Routing Tool

In Vue.js applications like ours, the most common tool for managing routes is **Vue Router**. It's a special library designed to handle all the navigation magic.

3. Defining Our Routes

All our application's routes are defined in a central file: `frontend/src/router/index.js`. This file acts as the master directory for our application's "rooms."

Here's a simplified look at how routes are defined:

```
// File: frontend/src/router/index.js (Simplified Excerpt)
import { createRouter, createWebHistory } from 'vue-router'

// Import the Vue Components (our "rooms")
import DashboardPage from '../views/user/dashboard.vue'
import User_Chap_Page from '../views/user/user_chapter_quiz_list.vue'
import Login from '../views/user/login.vue'
import NotFound from '../views/Error/404.vue' // For pages not found

const routes = [
  {
    path: '/login', // When the URL is /login
    name: 'login', // A friendly name for this route
    component: Login // Show the Login component
  },
  {
    path: '/dashboard',
    name: 'dashboard',
    component: DashboardPage
  },
  {
    path: '/dashboard/chapter/:chapter_id', // This is a dynamic route!
    name: 'user_chap_page',
    component: User_Chap_Page
  },
  {
    path: '/:catchAll(.*)', // A special route to catch any unmatched URLs
    name: 'notfound',
    component: NotFound
  }
];

// Create the router instance
const router = createRouter({
  history: createWebHistory(), // Tells Vue Router to use standard browser history
  routes // Pass our defined routes here
});

export default router; // Make the router available to our app
```

Explanation:

- `import { createRouter, createWebHistory } from 'vue-router'`: These are functions from the `vue-router` library that help us create our router.
- `import DashboardPage from ...`: We import all the Vue Components we want to use as "pages."
- `const routes = [...]`: This is an array (a list) of route objects. Each object defines one route.
  - `path`: The URL path.
  - `name`: A unique name for the route, which can be useful for programmatic navigation.
  - `component`: The Vue Component that should be displayed when this path is matched.
- **Dynamic Routes (`:chapter_id`)**: Look at `/dashboard/chapter/:chapter_id`. The colon `:` indicates that `:chapter_id` is a **placeholder** for any value. This means URLs like `/dashboard/chapter/123`, `/dashboard/chapter/abc`, or `/dashboard/chapter/my-great-story` will all match this route. The actual `123`, `abc`, or `my-great-story` part becomes available inside the `User_Chap_Page` component, allowing it to load the correct chapter.
- `createWebHistory()`: This makes our URLs look clean and standard (e.g., `myapp.com/dashboard` instead of `myapp.com/#/dashboard`).
- `export default router;`: This line makes our configured `router` object available for other parts of our application to use.

4. The `<router-view />` Placeholder

While we define many routes in `router/index.js`, where do the actual components appear on the screen? In our `frontend/src/App.vue` file (the main component of our entire Vue application), there's a special tag:

```
<!-- File: frontend/src/App.vue (Excerpt) -->
<template>
  <div id="app">
    <!-- This is where the magic happens! -->
    <router-view />
  </div>
</template>
```

Explanation:

- `<router-view />`: This is Vue Router's special component. It acts as a **placeholder**. When a URL matches a route, the `component` associated with that route (e.g., `DashboardPage` or `User_Chap_Page`) is dynamically rendered *inside* this `<router-view />` area.
- Think of `App.vue` as the main frame of a picture, and `<router-view />` as the empty space where different pictures (our Vue Components) can be swapped in and out based on the current URL.

5. Navigating Between Pages

There are two primary ways to move between routes:

- **Using `<router-link>` (For simple clicks):** This is Vue Router's equivalent of an HTML `<a>` tag.

```
<!-- Example using router-link in a template -->
<template>
  <div>
```

```
      <router-link to="/dashboard">Go to Dashboard</router-link>
      <router-link to="/login">Login Page</router-link>
    </div>
  </template>
```

When a user clicks on these, Vue Router intercepts the click and changes the component displayed in `<router-view />` without a full page reload.

- **Programmatic Navigation (`this.$router.push`)**: This is how we navigate using JavaScript code, often triggered by actions like a button click (e.g., "View Chapter" button), after a successful login, or when a form is submitted.

  Recall the `DashboardPage` from Chapter 2:

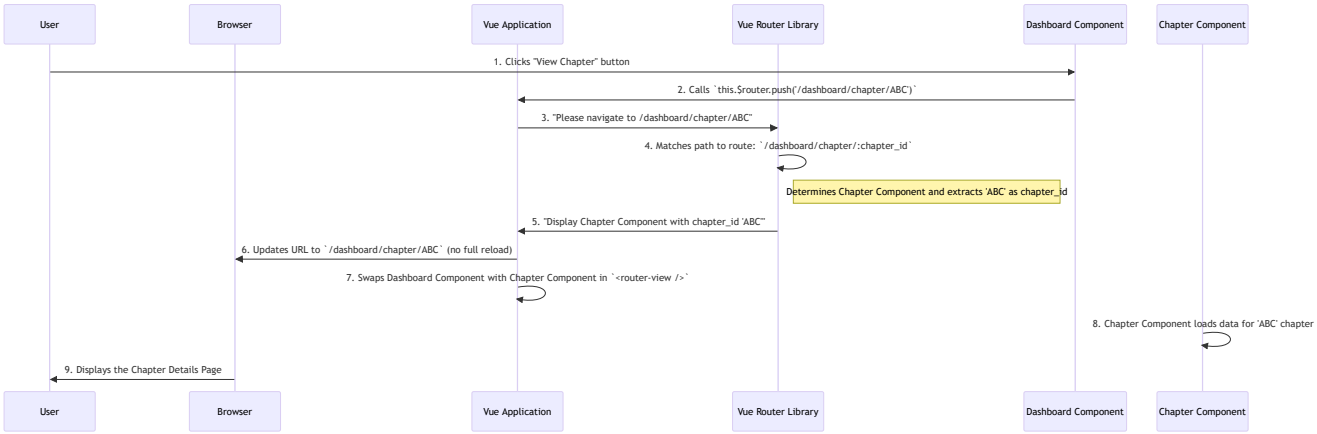
  ```
  // File: frontend/src/views/user/dashboard.vue (Excerpt)
  <script>
  export default {
      methods: {
          view_the_chap(chapter_id){
              // Using this.$router.push to navigate!
              // It builds the URL like "/dashboard/chapter/123"
              this.$router.push(`/dashboard/chapter/${chapter_id}`);
          },
          // ... other methods
      },
      // ...
  }
  </script>
  ```

  Explanation:

  - `this.$router`: This is a special object provided by Vue Router that gives us access to its navigation methods.
  - `this.$router.push(path)`: This method tells the router to "push" a new entry onto the browser's history stack, effectively navigating to the specified `path`. It's similar to typing a new URL into the address bar and hitting Enter, but without a full page refresh.
  - `${chapter_id}`: This is a JavaScript template literal, allowing us to easily insert the actual `chapter_id` into the URL path, creating a dynamic URL.

What Happens Under the Hood?

Let's visualize the journey when you click that "View Chapter" button on the Dashboard.



1. **User Clicks**: The user clicks the "View Chapter" button on the Dashboard.
2. **Method Call**: The `@click` directive on the button triggers the `view_the_chap(chapter_id)` method in the `DashboardPage` component.
3. **Programmatic Navigation**: Inside `view_the_chap`, `this.$router.push('/dashboard/chapter/${chapter_id}')` is called. This tells the overall Vue application that a navigation action is requested.
4. **Vue Router Takes Over**: The Vue application hands this navigation request to the Vue Router library.
5. **Route Matching**: Vue Router looks at its `routes` configuration (from `frontend/src/router/index.js`). It finds the best match for the path `/dashboard/chapter/ABC`, which is `/dashboard/chapter/:chapter_id`. It also cleverly extracts "ABC" as the value for `chapter_id`.
6. **Component Swap**: Vue Router then tells the main Vue application: "Hey, the component for this route is `User_Chap_Page` (our Chapter Component), and here's the `chapter_id` it needs."
7. **UI Update**: The Vue application then *removes* the `DashboardPage` component from the `<router-view />` slot and *inserts* the `User_Chap_Page` component in its place. The browser's URL is updated, but without a full page refresh.
8. **New Component Loads Data**: The newly loaded `User_Chap_Page` component, now knowing the `chapter_id` (e.g., "ABC"), can then make its own request to the Backend API Server (Flask `app.py`) to fetch the specific details for chapter "ABC" from the Database Models (SQLAlchemy ORM).
9. **User Sees New Page**: The user instantly sees the Chapter Details page, loaded with relevant information.

This seamless process is the power of frontend routing, making our application feel much more responsive and modern.

Summary

In this chapter, we learned that:

- **Frontend Routing** is like our application's internal GPS, defining which Frontend Views (Vue Components) should be displayed for different URLs.
- We use **Vue Router** to manage these routes.
- **Routes** are defined in `frontend/src/router/index.js`, mapping URL `path`s to specific `component`s.
- **Dynamic Routes** (like `/dashboard/chapter/:chapter_id`) allow parts of the URL to act as changeable parameters.
- The `<router-view />` tag in `frontend/src/App.vue` acts as a placeholder where matched components are rendered.

- We can navigate using HTML-like `<router-link>` tags or programmatically with `this.$router.push()` in our component's JavaScript code.

Understanding frontend routing is essential for creating a smooth and interactive user experience. In the next chapter, we'll delve into a critical aspect of any application: **Authentication & Session Management**, learning how we identify users and keep them logged in securely.

Next Chapter: Authentication & Session Management

Chapter 4: Authentication & Session Management

Welcome back! In our previous chapter, we mastered the art of **Frontend Routing**, learning how to seamlessly navigate between different "rooms" or pages in our application. We saw how clicking a "View Chapter" button instantly takes us to the correct chapter details without a full page reload.

But what if these "rooms" contain sensitive information, or actions only certain people should be able to perform? How does our application know *who* you are, whether you're allowed into a specific room, and how do you stay "recognized" as you move around the building without having to show your ID at every door?

This is where **Authentication & Session Management** steps in!

What's the Big Idea? (Motivation)

Imagine our application is a secure building, like a university.

- **Authentication** is like the **security check at the entrance**. It's the process of **verifying your identity**. Are you truly who you say you are? (e.g., You show your student ID and the guard checks it against a list, or you type your username and password, or for special access, you might use a one-time password (OTP)).
- **Authorization** is what happens *after* authentication. It's about **what you're allowed to do** once inside. A student can access classrooms, a professor can access the faculty lounge, and a librarian can access the book archives.
- **Session Management** is like getting a **digital "access card"** after you've been verified. Once you're identified at the main gate, you get this card (a token). Now, as you move between different rooms (pages or features) in the building, you just flash your card instead of going through the full ID check every single time. This makes your experience smooth and efficient.

This system is the security guard for our entire `MAD-2-Project-iitm` application. It handles everything from allowing new users to sign up, verifying their identity when they log in (using passwords or OTPs for admins), and then issuing a secure digital "pass" (a **JWT token**) that allows them to access different parts of the website without repeatedly logging in. It ensures only authorized individuals can perform actions.

Our goal in this chapter is to understand how a user registers, logs in, and then continues to be recognized by the application as they browse. We'll also briefly touch upon how an admin's login process is a bit different.

Key Concepts Explained

Let's break down the core ideas behind how our application manages its security.

Concept	Analogy	What it means for our app
Authentication	Showing your ID at the gate	Verifying a user's identity (e.g., username/password for students, email/OTP for admins).
Authorization	What rooms your ID allows you into	Determining what actions a user is allowed to perform after they're logged in (e.g., a student can take quizzes, an admin can manage chapters).

Concept	Analogy	What it means for our app
Session Management	The digital "access card" you get	Keeping a user logged in across multiple pages or visits without requiring them to re-enter credentials every time.
Password Hashing	A locked box that can't be opened	Converting a user's plain-text password into a scrambled, irreversible string before storing it. If someone steals our database, they can't see actual passwords.
JWT (JSON Web Token)	Your secure, temporary "digital pass"	A compact, self-contained piece of information that safely transmits user identity and permissions between the frontend and backend. It's signed to prevent tampering.
Cookies	The "wallet" where your digital pass is stored	Small pieces of data stored by your web browser. This is where we keep the JWT token so the browser can send it with every request.

User Registration: Getting Your First ID

Before anyone can log in, they need to create an account. This is the **registration** process.

How it Works (Frontend - `register.vue`):

When a user visits the registration page (`/register`), they fill out a form with their desired username, email, and password.

```
<!-- File: frontend/src/views/user/register.vue (Simplified Excerpt) -->
<template>
    <div class="register-container">
        <form class="registration-form">
            <input type="text" v-model="username" />
            <input type="email" v-model="email" />
            <input type="password" v-model="password" />
            <input type="password" v-model="confirmPassword" />
            <button type="button" @click="register">Register</button>
        </form>
    </div>
</template>
```

The important part is the JavaScript logic within the `<script>` section when the "Register" button is clicked:

```
// File: frontend/src/views/user/register.vue (Simplified Excerpt)
<script>
import axios from 'axios';
export default {
    data() {
        return { username: "", email: "", password: "", confirmPassword: "" };
```

```
    },
    methods: {
        async register() {
            // 1. Basic validation (passwords match, all fields filled)
            if (this.password !== this.confirmPassword) {
                alert("Passwords do not match."); return;
            }
            // 2. Prepare data to send to the backend
            const reg_data = new FormData();
            reg_data.append("username", this.username);
            reg_data.append("email", this.email);
            reg_data.append("password", this.password); // Plain password sent
(will be hashed by backend)

            try {
                // 3. Send registration data to the backend API
                const response = await
axios.post(`http://127.0.0.1:5000/register`, reg_data);
                alert(`You are registered: ${response.data.message}`);
                // 4. On successful registration, navigate to the login page
                this.$router.push(`/login`); // Go to [Frontend Routing]
(03_frontend_routing_.md)
            } catch (error) {
                alert(`Error registering: ${error.response.data.message}`);
            }
        }
    }
};
</script>
```

Explanation:

- The `register` method is called when the user clicks the "Register" button.
- It first performs simple checks (like ensuring passwords match).
- Then, it gathers the user's input (`username`, `email`, `password`) and bundles it into `FormData`.
- `axios.post` is used to send this data to our backend server at the `/register` address.
- If the backend successfully creates the user, an alert confirms it, and the user is then redirected to the `/login` page using `this.$router.push`.

User Login: Showing Your Digital Pass

Once registered, a user can log in to get their digital "access card" (JWT token).

How it Works (Frontend - `login.vue`):

The login page (`/login`) is where users enter their username (or email) and password.

```
<!-- File: frontend/src/views/user/login.vue (Simplified Excerpt) -->
<template>
    <div class="login-container">
        <form class="login-form">
```

```
            <input type="text" v-model="username" />
            <input type="password" v-model="password" />
            <button type="button" @click="login">Login</button>
        </form>
    </div>
</template>
```

Here's the key part of the JavaScript:

```
// File: frontend/src/views/user/login.vue (Simplified Excerpt)
<script>
import axios from 'axios';
import VueCookies from 'vue-cookies'; // Tool to manage web cookies

export default {
    data() {
        return { username: "", password: "" };
    },
    methods: {
        async login() {
            // 1. Basic validation (ensure fields are filled)
            if (!this.username || !this.password) {
                alert("Please fill in both fields."); return;
            }
            // 2. Prepare data for backend
            const login_data = new FormData();
            login_data.append("identifier", this.username);
            login_data.append("password", this.password);

            try {
                // 3. Send login request to the backend
                const response = await axios.post(`http://127.0.0.1:5000/login`,
login_data);

                // 4. IMPORTANT: Store the received JWT token in a cookie
                VueCookies.set('access_token', response.data.token, {
                    expires: '4d', // Token valid for 4 days
                    path: '/',     // Available across the entire website
                    samesite: 'Lax', // Security setting
                    secure: false    // Set to true in production for HTTPS
                });

                // 5. Redirect to the dashboard
                this.$router.push(`/dashboard/`); // Go to [Frontend Routing]
(03_frontend_routing_.md)
            } catch (error) {
                alert(`Login failed! Check credentials.`);
            }
        }
    }
};
</script>
```

Explanation:

- The `login` method collects the user's input.
- `axios.post` sends this to the backend's `/login` endpoint.
- If the login is successful, the backend sends back a `response.data.token` (our JWT digital pass!).
- `VueCookies.set()` is then used to store this token in the user's browser as a cookie. This cookie will automatically be sent with almost every subsequent request to our backend, so the backend always knows who the user is.
- Finally, the user is redirected to the dashboard.

Admin Login: OTP Special Access

Admin users have a slightly different, more secure login process using a One-Time Password (OTP) sent to their email. This is a common practice for high-privilege accounts.

How it Works (Frontend - `admin_login.vue`):

The admin login process is divided into two steps:

1. **Request OTP:** Admin enters their email.
2. **Verify OTP:** Admin enters the OTP received in their email.

```
<!-- File: frontend/src/views/admin/admin_login.vue (Simplified Excerpt) -->
<template>
    <div class="card-login">
        <!-- Step 1: Email Input -->
        <div v-if="step === 1">
            <input type="email" v-model="email" />
            <button type="submit" @click="requestOtp">Send OTP</button>
        </div>

        <!-- Step 2: OTP Input -->
        <div v-if="step === 2">
            <p>An OTP has been sent to <strong>{{ email }}</strong></p>
            <input type="text" v-model="otp" maxlength="6" />
            <button type="submit" @click="verifyOtpAndLogin">Verify &
Login</button>
        </div>
    </div>
</template>
```

The JavaScript handles the multi-step process:

```
// File: frontend/src/views/admin/admin_login.vue (Simplified Excerpt)
<script>
import axios from 'axios';
import VueCookies from 'vue-cookies';
```

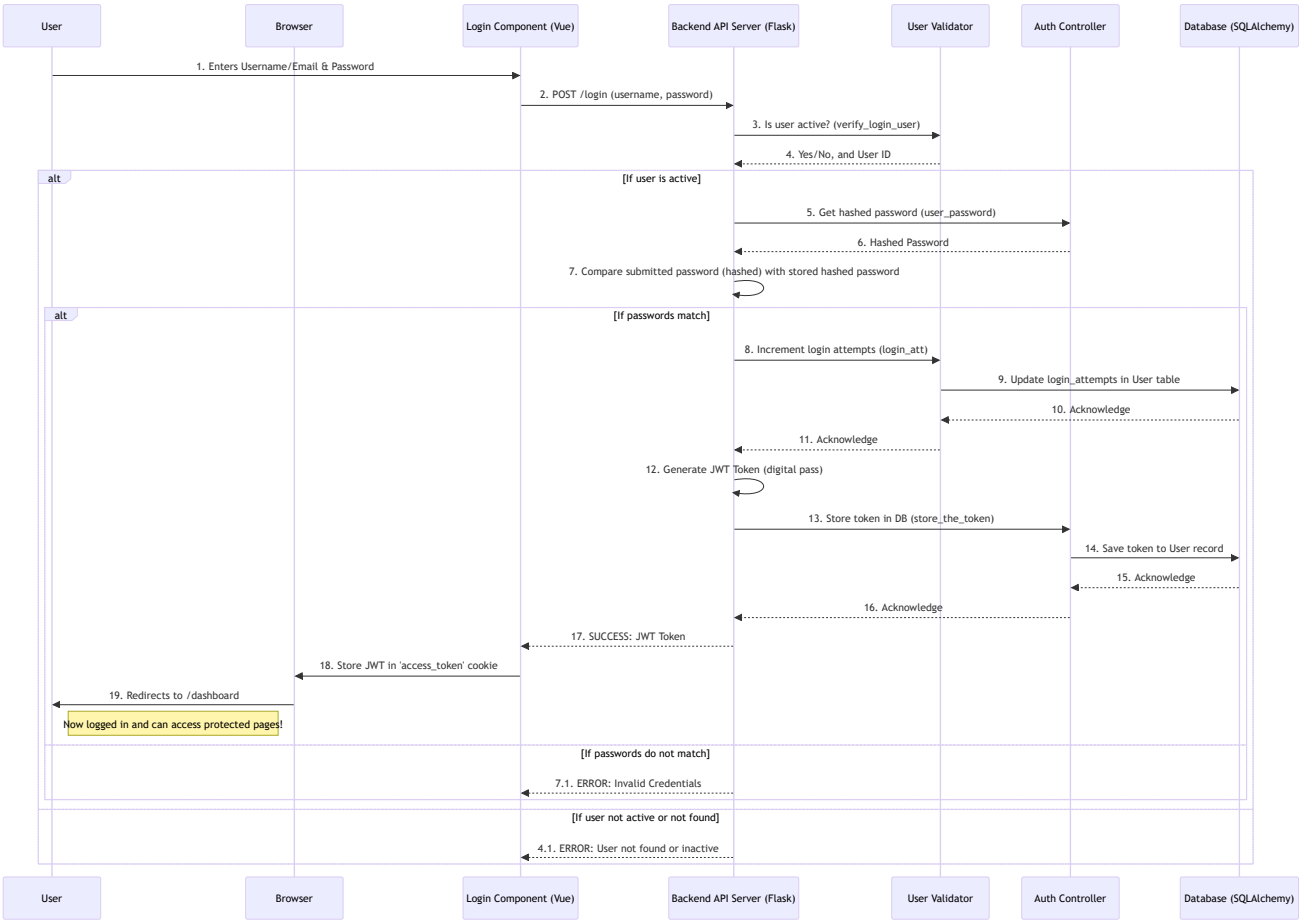
```
export default {
    data() { return { step: 1, email: "", otp: "" }; },
    methods: {
        async requestOtp() {
            if (!this.email) return;
            const formData = new FormData(); formData.append("email", this.email);
            try {
                // 1. Send request to backend to generate and send OTP
                await axios.post(`http://127.0.0.1:5000/api/request_otp`,
formData);
                alert(`An OTP has been sent to ${this.email}.`);
                this.step = 2; // Move to OTP input step
            } catch (error) {
                alert(`Failed to send OTP.`);
            }
        },
        async verifyOtpAndLogin() {
            if (!this.otp || this.otp.length !== 6) return;
            const loginData = new FormData();
            loginData.append("email", this.email); loginData.append("otp",
this.otp);
            try {
                // 2. Send OTP to backend for verification
                const response = await
axios.post(`http://127.0.0.1:5000/api/verify_otp`, loginData);
                // 3. Store admin token on success
                VueCookies.set('admin_token', response.data.admin_token, {
expires: '1d', path: '/' });
                alert('Admin login successful!');
                this.$router.push(`/admindashboard`); // Redirect to admin
dashboard
            } catch (error) {
                alert(`Login failed! Invalid OTP.`);
            }
        },
        goBack() { this.step = 1; this.otp = ""; }
    }
};
</script>
```

Explanation:

- `requestOtp` sends the admin's email to a backend endpoint (`/api/request_otp`) which generates an OTP and sends it via email. Once successful, the frontend `step` changes to `2`.
- `verifyOtpAndLogin` sends both the email and the entered OTP to another backend endpoint (`/api/verify_otp`).
- If the OTP is correct, the backend responds with an `admin_token`, which is then stored in a cookie (`admin_token` instead of `access_token` for clarity).
- The admin is then redirected to their specific dashboard.

What Happens Under the Hood?

Let's trace the journey of a user logging in and staying authenticated.



Backend Deep Dive: The Security Engine

The real work of authentication and session management happens on the backend. Here's a look at some crucial parts:

1. User Registration Logic (`backend/utils/user_validator.py` & `backend/controllers/user/auth.py`)

When a user tries to register, the backend first ensures that the username or email isn't already taken.

```
# File: backend/utils/user_validator.py (Simplified Excerpt)
from models.model import User # Our blueprint from Chapter 1

def verify_pre_user(username, email):
    # Query the database to see if a user with this username OR email already
exists
    user = User.query.filter_by(username=username).first()
    if user: # If user exists
        return False # Cannot register, username taken
    user = User.query.filter_by(email=email).first()
    if user: # If user exists
        return False # Cannot register, email taken
    return None # OK to register!
```

Explanation:

- `User.query.filter_by(...)` is an SQLAlchemy command (from Chapter 1: Database Models) to find a user based on `username` or `email`.
- If a user is found, it means the username or email is already in use, and registration should fail.

Once validated, the user is created. Crucially, the password is **hashed** (scrambled) before being saved.

```
# File: backend/controllers/user/auth.py (Simplified Excerpt)
from models.model import User, db # Our blueprint and database object
# (Assume 'generate_password_hash' function is imported here)

def register_user(id, username, email, plain_password):
    # Hash the password for security before storing it
    hashed_password = generate_password_hash(plain_password)

    # Create a new User object and add to database
    new_user = User(uuid=id, username=username, email=email,
password=hashed_password)
    db.session.add(new_user)
    db.session.commit() # Save changes to the database
    return True
```

Explanation:

- `generate_password_hash(plain_password)`: This is a function (usually from a library like `werkzeug.security`) that takes the plain password the user typed and turns it into a long, scrambled string. This string is irreversible, meaning you can't get the original password back from the hash.
- The `hashed_password` is then stored in the `User` object (our Database Model) and committed to the database.

2. User Login Logic (`backend/utils/user_validator.py` & `backend/controllers/user/auth.py`)

When a user attempts to log in, the backend performs several checks:

```
# File: backend/utils/user_validator.py (Simplified Excerpt)
from models.model import User

def verify_login_user(identifier):
    # Find user by username OR email
    user = User.query.filter((User.username == identifier) | (User.email ==
identifier)).first()
    if user:
        if user.is_active: # Check if user account is active
            return True  # User exists and is active
        return False # User exists but is not active
    return None # User does not exist at all

def user_id_function(identifier):
```

```
    # Get the user's unique ID
    user = User.query.filter((User.username == identifier) | (User.email ==
identifier)).first()
    return user.uuid

def login_att(user_id):
    # Track login attempts (for security, e.g., to block after too many failures)
    user = User.query.filter_by(uuid=user_id).first()
    if user:
        attempt = user.login_attempts # Get current attempts
        user.login_attempts += 1     # Increment attempts
        db.session.commit()          # Save to database
        return attempt
    return None
```

Explanation:

- `verify_login_user`: Checks if a user exists with the provided username or email, and if their account is active.
- `user_id_function`: Retrieves the user's unique identifier (`uuid`).
- `login_att`: Increments a counter for login attempts for that user. This is a basic security measure to prevent brute-force attacks (where someone tries many passwords).

Next, the backend needs the stored hashed password to compare it with the one the user just typed:

```
# File: backend/controllers/user/auth.py (Simplified Excerpt)
from models.model import User

def user_password(identifier):
    user = User.query.filter((User.username == identifier) | (User.email ==
identifier)).first()
    if user:
        return user.password  # Return the hashed password from the database
    return None
```

Explanation:

- This function simply fetches the `User` object and returns its `password` field, which contains the previously *hashed* password.

The backend then uses another function (e.g., `check_password_hash` from `werkzeug.security`) to compare the *newly submitted* password (after it's also hashed) with the *stored hashed* password. If they match, the user is authenticated!

Finally, upon successful login, the backend generates and stores the JWT token:

```
# File: backend/controllers/user/auth.py (Simplified Excerpt)
from models.model import User, db
```

```
def store_the_token(user_id, token):
    # Find the user by their ID
    user = User.query.filter_by(uuid = user_id).first()
    if user:
        user.access_token = token # Store the new JWT token in the user's record
        db.session.commit()       # Save this change
        return True
    return False

def check_token_in_user(token):
    # Used for verifying incoming requests with tokens
    user = User.query.filter_by(access_token = token).first()
    if user:
        return user.uuid # Return the user's ID if the token is valid and found
    return None
```

Explanation:

- `store_the_token`: After a successful login, a new JWT token is created (this happens in our `app.py` or an API controller, which we'll see in Chapter 5: Backend API Server and Chapter 6: API Controllers). This token is then saved in the `access_token` column of the `User` record in our database. This allows us to revoke tokens later if needed.
- `check_token_in_user`: When a user makes a request to a protected part of the application (e.g., fetching their dashboard data), the backend takes the `access_token` from their cookie and uses this function to quickly check if that token exists in the database and belongs to a valid user.

3. Admin Login Logic (OTP) (`backend/controllers/admin/auth.py`)

Admin login is similar but uses OTPs for added security.

```
# File: backend/controllers/admin/auth.py (Simplified Excerpt)
import random # For generating OTP
import smtplib # For sending email
from models.model import Admin, db # Our Admin blueprint from Chapter 1

def find_admin(email):
    # Find the admin by email
    admin = Admin.query.filter_by(admin_email=email).first()
    return admin

def send_otp_for_admin(admin_object):
    try:
        otp_code = str(random.randint(100000, 999999)) # Generate a 6-digit OTP
        admin_object.admin_token = otp_code # Temporarily store OTP in admin_token
field
        db.session.commit() # Save to database

        # Code to send OTP email (using smtplib, omitted details)
        # ... connect to mail server, send email with otp_code ...
        return True
```

```
        except Exception as e:
            db.session.rollback()
            return False

    def actual_otp(email):
        # Retrieve the stored OTP for verification
        admin = Admin.query.filter_by(admin_email=email).first()
        if admin:
            return admin.admin_token # This is where the OTP is temporarily stored
        return None
```

Explanation:

- `find_admin`: Checks if an admin account exists for the given email.
- `send_otp_for_admin`: If the admin email is valid, a random 6-digit OTP is generated. This OTP is then *temporarily saved* in the `admin_token` field of the `Admin` model (yes, we reuse the token field for OTP in this simplified setup!) and then sent to the admin's email.
- `actual_otp`: When the admin submits the OTP, this function retrieves the *stored* OTP from the database for comparison. If the submitted OTP matches the stored one, the admin is authenticated. Then, a proper JWT admin token is generated and sent back to the frontend to be stored in a cookie.

This comprehensive system of user verification, password security, and session management is fundamental to building any secure web application.

Summary

In this chapter, we unpacked the critical concepts of **Authentication & Session Management**:

- **Authentication** verifies who you are (like an ID check).
- **Authorization** determines what you can do.
- **Session Management** uses a digital "pass" (JWT token stored in a cookie) to keep you logged in.
- We saw how **user registration** works by collecting information and saving passwords securely through **hashing**.
- **User login** involves verifying credentials and then issuing a JWT token, which the browser stores in a **cookie**.
- **Admin login** uses a more secure, multi-step process with **OTP** sent to email.
- The backend logic handles these processes, from validating user input and hashing passwords to storing and checking tokens.

Understanding these security foundations is paramount. In the next chapter, we'll see how all these pieces, from database models to frontend views and authentication logic, come together in our **Backend API Server (Flask `app.py`)**, which acts as the central hub for our entire application.

Next Chapter: Backend API Server (Flask `app.py`)

Chapter 5: Backend API Server (Flask `app.py`)

Welcome back! In our previous chapter, we learned about **Authentication & Session Management**, how users register, log in, and stay recognized by our application using secure digital passes (JWT tokens). This security is crucial, but where do these login requests go? And how does the application then *use* that security to give users their personalized experience?

This is where the **Backend API Server**, specifically our `app.py` file, comes into play!

What's the Big Idea? (Motivation)

Imagine our application is a bustling restaurant.

- The **Frontend (Vue Components)** (from Chapter 2) is like the beautiful dining area where customers (users) sit and interact with the menu (UI).
- The **Database Models** (from Chapter 1) are like the pantry and ingredient lists, perfectly organized.
- **Authentication & Session Management** (from Chapter 4) is like the bouncer and the reservation system, ensuring only authorized guests get in and find their seats.

But who takes the orders from the customers, tells the kitchen what to cook, brings the food out, and handles all the behind-the-scenes work? That's the **Backend API Server**! In our project, this is primarily managed by the `app.py` file using a powerful Python tool called **Flask**.

Flask `app.py` is the **central command center** of our entire application, like the brain of the system. It's constantly listening for requests from the frontend (your browser) – whether you're trying to log in, view your dashboard, take a quiz, or update your profile.

Once it receives a request, `app.py` acts like a **dispatcher**. It processes the request, potentially checks your authentication (using the tokens we learned about in Chapter 4), calls the appropriate "workers" (called **controllers**, which we'll cover in Chapter 6) to do the heavy lifting (like getting data from the database), and then sends back the required data to your browser.

It also cleverly manages other background services like the task queue (Celery/Redis, which we'll see in Chapter 7), starting them automatically to keep the application running smoothly without extra manual steps.

Our goal in this chapter is to understand how `app.py` serves as this central hub, bringing all the different parts of our application together. We'll specifically look at how it handles a request to display a user's dashboard after they've logged in.

Key Concepts Explained

Let's break down the core ideas behind our Backend API Server.

Concept	Analogy	What it means for our app (`app.py`)
API (Application Programming Interface)	A waiter taking your order and telling the kitchen how to prepare it	A set of rules and definitions that allows the frontend (your browser) to communicate with the backend (our server) to request data or perform actions.

Concept	Analogy	What it means for our app (`app.py`)
Server	The restaurant's kitchen and front desk	A program (`app.py` in our case) that runs on a computer, waiting for requests from clients (browsers) and sending back responses.
Flask	A specialized chef (like a grill master)	A lightweight Python framework that helps us easily build web servers and APIs. It handles the basic structure of receiving requests and sending responses.
Route / Endpoint	Specific dishes on the menu with unique numbers	A specific URL path (e.g., `/login`, `/dashboard/chapter_n_quiz`) that the Flask server listens for. Each route corresponds to a specific action or data.
Request & Response	Customer ordering & waiter bringing food	A **Request** is information sent from the frontend to the backend (e.g., "Login with this username/password"). A **Response** is the information sent back from the backend to the frontend (e.g., "Login successful, here's your token!").
JSON (JavaScript Object Notation)	A standardized order slip or recipe book	A common, human-readable format for sending data between the frontend and backend. It's like a universal language for data.

Setting Up Our Flask Server (`app.py`)

Our `backend/app.py` file is where all the main server setup and route definitions live.

1. Basic Flask Application Setup

The first few lines in `app.py` set up the Flask application itself and connect it to our configurations and database.

```
# File: backend/app.py (Excerpt)
from flask import Flask, request, jsonify # Import Flask and tools for
requests/responses
from config import Config # Our application settings
from database import db # Our database connection from Chapter 1
from flask_cors import CORS # For allowing frontend to talk to backend
from flask_caching import Cache # For speeding up responses
import bcrypt # For password hashing
import jwt # For JWT tokens
import os
from datetime import datetime, timedelta, timezone

# --- Import all our "worker" functions (controllers) ---
from controllers.admin.create import cre_level # Example import
from controllers.user.auth import check_token_in_user, register_user # Example
import
# ... many more imports for different features ...
```

```
# Create our Flask application!
app = Flask(__name__)

# Load all settings from our Config file
app.config.from_object(Config)

# Connect Flask app to our database object (from Chapter 1)
db.init_app(app)

# Setup caching for faster responses
cache = Cache(app)

# Enable Cross-Origin Resource Sharing (CORS)
# This allows our frontend (running on one address) to talk to our backend
(running on another)
if app.config['CORS_ENABLED']:
    CORS(app, origins=app.config['CORS_ORIGINS'], supports_credentials=True)

# Register custom error handling functions
from middlewares.error_handlers import register_error_handlers
register_error_handlers(app)
```

Explanation:

- `app = Flask(__name__)`: This line creates our Flask application instance. Think of it as opening our restaurant doors.
- `app.config.from_object(Config)`: This loads all our important settings (like database connection details, secret keys, etc.) from the `Config` class defined in `backend/config.py`. This keeps sensitive information and configuration tidy.
- `db.init_app(app)`: This connects our Flask application to our `db` object (from `backend/database.py`), which is our gateway to interacting with the database using SQLAlchemy (as discussed in Chapter 1: Database Models).
- `cache = Cache(app)`: Initializes our caching system. Caching stores frequently requested data temporarily so the server doesn't have to fetch it from the database every single time, making responses much faster!
- `CORS(app, ...)`: **CORS (Cross-Origin Resource Sharing)** is a security feature. When your frontend (e.g., running on `localhost:8080`) tries to talk to your backend (e.g., running on `localhost:5000`), browsers normally block this for security. CORS settings here tell the browser, "It's okay, this frontend is allowed to talk to me!"
- `register_error_handlers(app)`: This sets up custom messages for common errors (like "Page Not Found" or "Unauthorized"), making our application more user-friendly.

2. The Heartbeat: Running the Server and Background Tasks

At the very bottom of `app.py`, there's a special block that makes our Flask application start running and also ensures our background services (like Celery) are launched.

```
# File: backend/app.py (Excerpt)
# ... (previous code) ...
```

```
# List to keep track of our background processes
processes = []

def start_background_services():
    """A helper function to start the Celery services."""
    print("--- Starting Background Services (Main Process Only) ---")

    # Command to start the Celery Worker, which executes tasks
    worker_cmd = ["celery", "-A", "app.celery", "worker", "--pool=solo", "--
loglevel=info"]
    worker_p = subprocess.Popen(worker_cmd)
    processes.append(worker_p)
    print(f"Celery Worker started with PID: {worker_p.pid}")

    # Command to start the Celery Beat, which schedules tasks
    beat_cmd = ["celery", "-A", "app.celery", "beat", "--loglevel=info"]
    beat_p = subprocess.Popen(beat_cmd)
    processes.append(beat_p)
    print(f"Celery Beat started with PID: {beat_p.pid}")


def stop_background_services():
    """A helper function to stop the services when press CTRL+C."""
    print("\n--- Stopping all background services... ---")
    for p in processes:
        p.terminate()  # Ask the process to stop gracefully
    print("--- Shutdown complete. ---")


# This is the main entry point of our script.
if __name__ == '__main__':
    # This crucial check prevents Celery from starting multiple times
    # when Flask's reloader is active during development.
    if os.environ.get("WERKZEUG_RUN_MAIN") != "true":
        with app.app_context():
            print("--- Checking and creating database tables... ---")
            # This line is key: it creates the .sqlite3 file and all your tables
            # based on your SQLAlchemy models (from Chapter 1).
            db.create_all()
            print("--- Database is ready. ---")

        # Start the Celery Worker and Beat in the background.
        start_background_services()

        # Register cleanup to run ONLY when the main process exits (e.g., CTRL+C).
        atexit.register(stop_background_services)

    # Start the Flask web server. 'debug=True' enables auto-reloading.
    print("--- Starting Flask Web Server (Reloader Active) ---")
    print(f"Visit http://127.0.0.1:5000 or on port 5000 in your browser.")
    app.run(debug=True, port=5000)
```

Explanation:

- `if __name__ == '__main__':`: This is a standard Python idiom. It means the code inside this block will *only* run when you directly run `app.py` (e.g., `python app.py`) and not when it's imported as a module elsewhere.
- `db.create_all()`: This is a powerful SQLAlchemy command! It looks at all your Database Models (SQLAlchemy ORM) and creates the corresponding tables in your database if they don't already exist. This is why you run `app.py` first to set up the database.
- `start_background_services()`: This function uses Python's `subprocess` module to run commands that start our **Celery worker** and **Celery beat** in the background.
    - **Celery Worker**: This process waits for tasks (like sending emails or generating reports) that the main Flask app wants to run in the background without blocking the user's request.
    - **Celery Beat**: This process is like a scheduler. It reads our `BEAT_SCHEDULE` from `config.py` and tells the Celery worker *when* to run certain tasks automatically (e.g., "send a daily summary email at 6 PM").
    - We'll explore Celery in more detail in Chapter 7: Asynchronous Task Queue (Celery/Redis).
- `atexit.register(stop_background_services)`: This ensures that when you stop the Flask server (e.g., by pressing `Ctrl+C`), the `stop_background_services()` function is called, which gracefully shuts down the Celery processes.
- `app.run(debug=True, port=5000)`: This line starts the Flask web server!
    - `debug=True`: This is great for development. If you make changes to your Python code, Flask will automatically restart the server for you.
    - `port=5000`: This tells Flask to listen for requests on port 5000 of your computer (so you access it in your browser at `http://127.0.0.1:5000`).

Example Use Case: Fetching Dashboard Data

Let's revisit our dashboard example from Chapter 2: Frontend Views (Vue Components). After a user logs in, the `DashboardPage` Vue Component (on the frontend) needs to fetch the list of chapters and quizzes relevant to that user. How does `app.py` handle this request?

The Route Definition

Every action or piece of data the frontend needs must have a corresponding "route" (or "endpoint") defined in `app.py`. This is like adding an item to our restaurant's menu.

```
# File: backend/app.py (Excerpt)
# ... (imports and app setup) ...

# This is a special decorator provided by Flask.
# It tells Flask: "When a GET request comes to '/dashboard/chapter_n_quiz', run
the function below!"
@app.route('/dashboard/chapter_n_quiz', methods=['GET'])
def user_dashboard_chapter():
    # 1. Get the authentication token from the request.
    token = request.args.get('token')

    # 2. Verify the token to ensure the user is logged in (from Chapter 4).
    payload, error_response, status_code = check_token(token)
```

```
    if payload is None: # If token is missing, expired, or invalid
        return jsonify({"message": f"{error_response}"}), status_code

    # 3. Extract the user's ID from the valid token.
    user_id = payload['user_id']

    # 4. Call our "worker" functions (controllers) to get data from the database.
    #    'user_selected_chap' checks for user's preferred chapters.
    user_favorite_chapter = user_selected_chap(user_id)

    if user_favorite_chapter:
        # If user has preferred chapters, get quizzes related to them.
        quiz_list = q1_list(user_favorite_chapter, user_id)
        return jsonify({ # 5. Send back the data as JSON.
            "message": "User has some bookmarked chapter",
            "chapters": user_favorite_chapter,
            "quiz": quiz_list
        }), 200
    else:
        # If no preferred chapters, get all chapters from user's selected
subjects.
        chapters_from_sub = list_all_chapters_from_user_sel_sub(user_id)
        quiz_list = q2_list(chapters_from_sub, user_id)
        return jsonify({ # 5. Send back the data as JSON.
            "message": "User has no bookmarked chapter, but here are all chapters
from selected subjects",
            "chapters": chapters_from_sub,
            "quiz": quiz_list
        }), 200
```

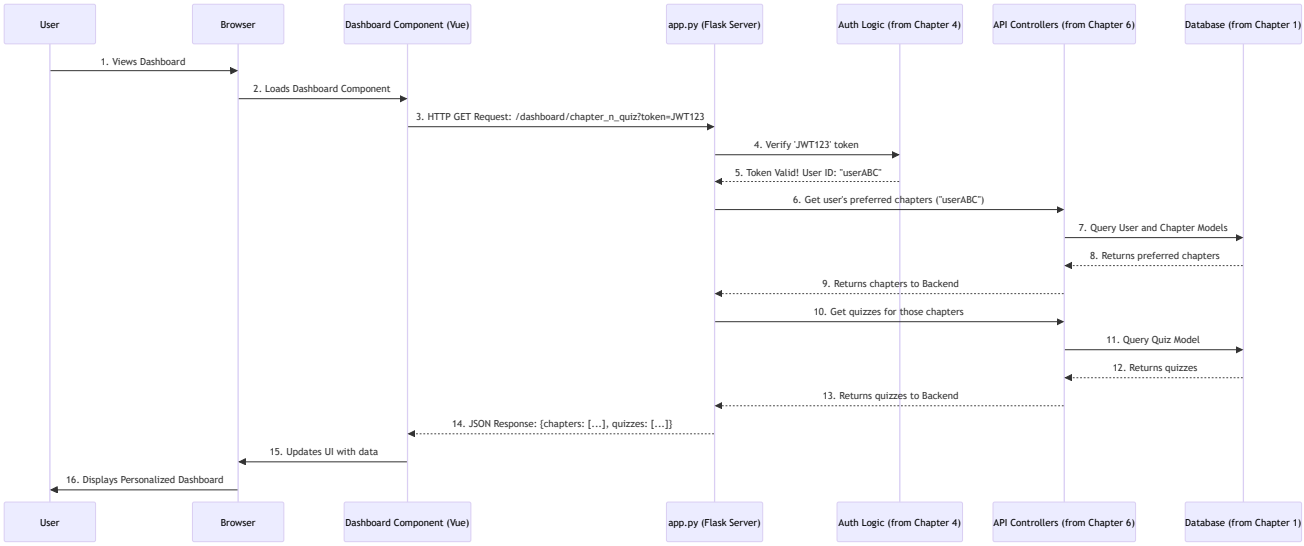
Explanation:

- `@app.route('/dashboard/chapter_n_quiz', methods=['GET'])`: This is a **decorator**. It tells Flask that the `user_dashboard_chapter` function should be executed whenever a `GET` request is made to the `/dashboard/chapter_n_quiz` URL.
- `request.args.get('token')`: This line accesses the `token` (our JWT digital pass) that the frontend sends in the URL's query parameters.
- `payload, error_response, status_code = check_token(token)`: This calls a helper function (also defined in `app.py` and using concepts from Chapter 4: Authentication & Session Management) to verify if the token is valid and extract the `user_id` from it. If the token is invalid, it immediately returns an error.
- `user_id = payload['user_id']`: Once the token is verified, we get the `user_id`. This is how the server knows *which user's* dashboard data to fetch.
- `user_selected_chap(user_id)`, `q1_list(...)`, `list_all_chapters_from_user_sel_sub(user_id)`, `q2_list(...)`: These are calls to our "worker" functions (our **API Controllers**), which contain the actual logic to fetch data from the database. We'll dive deep into these in Chapter 6: API Controllers (Business Logic).
- `return jsonify({...}), 200`: After gathering all the necessary data, the Flask function bundles it into a **JSON** response (a standard data format) and sends it back to the frontend.

- `200` is an **HTTP status code**, meaning "OK" or "Success". Other common codes are `401` (Unauthorized), `404` (Not Found), `500` (Internal Server Error).

What Happens Under the Hood?

Let's visualize how a dashboard data request travels through our application, orchestrated by `app.py`.



1. **User Views Dashboard**: The user is already logged in and navigates to the dashboard (e.g., `/dashboard`).
2. **Dashboard Component Loads**: The frontend's `DashboardPage` component starts up.
3. **Frontend Requests Data**: The `DashboardPage` component (using `axios` from Chapter 2) sends an HTTP `GET` request to `http://127.0.0.1:5000/dashboard/chapter_n_quiz`, including the JWT token in the URL.
4. **Flask Receives Request**: Our `app.py` Flask server receives this request at the `@app.route` for `/dashboard/chapter_n_quiz`.
5. **Token Verification**: `app.py` first calls the `check_token` function (part of Authentication & Session Management) to ensure the token is valid and the user is authenticated. If not, it sends an error immediately.
6. **User ID Extraction**: From the valid token, `app.py` extracts the `user_id`. This tells the server *who* is making the request.
7. **Calling API Controllers**: `app.py` then calls the relevant "worker" functions from our `controllers` directory (e.g., `user_selected_chap`, `q1_list`). These functions contain the specific business logic.
8. **Database Interaction**: The API Controllers interact with the `db` object (from Chapter 1: Database Models) to fetch the required chapter and quiz data from the database.
9. **Data Returns to `app.py`**: The data is sent back from the Database, through the API Controllers, to `app.py`.
10. **Flask Sends JSON Response**: `app.py` takes the fetched data, formats it as JSON, and sends it back as an HTTP response to the frontend.
11. **Frontend Updates UI**: The `DashboardPage` component receives the JSON data, updates its internal `data` (as seen in Chapter 2), and Vue automatically updates the HTML displayed to the user.
12. **User Sees Dashboard**: The user sees their personalized dashboard with the correct chapters and quizzes.

This entire flow, orchestrated by `app.py`, is how our application provides a dynamic and interactive experience.

Summary

In this chapter, we learned that:

- The **Backend API Server**, powered by Flask in `app.py`, is the **central command center** of our application.
- It listens for requests from the frontend at specific **routes** (or **endpoints**).
- It handles **requests** (like a user fetching their dashboard data) and sends back **responses** (often in **JSON** format).
- `app.py` orchestrates the flow: verifying **authentication tokens** (from Chapter 4), calling specialized "worker" functions (**API Controllers** for business logic, to be covered in Chapter 6), and interacting with the **database** (as defined by Chapter 1: Database Models).
- It also handles crucial setup like loading configuration, enabling CORS, setting up caching, and automatically starting background services like Celery.

Understanding `app.py` is key to seeing how all the pieces of our application connect and communicate. In the next chapter, we'll dive deeper into those "worker" functions, the **API Controllers**, and see how they contain the specific business logic that processes our data.

Next Chapter: API Controllers (Business Logic)

Chapter 6: API Controllers (Business Logic)

Welcome back! In our previous chapter, we explored how our **Backend API Server (Flask `app.py`)** acts as the central command center, listening for requests and coordinating responses. Think of `app.py` as the restaurant's main dispatcher, taking orders from customers (your browser).

But imagine a busy restaurant kitchen. The dispatcher takes the order, but they don't actually *cook* the food themselves. They hand it off to specialized chefs or departments – maybe one chef handles salads, another handles grilling, and a third handles desserts. Each of these specialists knows exactly *how* to prepare their part of the meal.

This is exactly the problem that **API Controllers** solve in our application!

What's the Big Idea? (Motivation)

While `app.py` (our dispatcher) receives all incoming requests, it would get very messy if it had to handle *all* the detailed steps for *every single request*. For example, if a user wants to "view their quiz summary," `app.py` shouldn't be burdened with figuring out how many quizzes were attempted, calculating average scores, or listing scores per subject. That's a lot of complex "cooking"!

This is where **API Controllers** come in. They are like these **specialized chefs or departments** in our backend.

- When `app.py` receives a request (e.g., "get user summary"), it hands it over to the **relevant API Controller**.
- This controller then contains all the **specific instructions and logic** (the "recipe") to fulfill that particular request. It knows how to talk to the Database Models (SQLAlchemy ORM), perform calculations, and gather all the necessary data.
- Once the controller has done its job, it sends the prepared results back to `app.py`, which then sends them to the frontend.

Our goal in this chapter is to understand how these controllers organize our backend code, making it clean and efficient. We'll specifically look at how the "user quiz summary" feature is handled by an API Controller.

Key Concepts Explained

Let's break down the core ideas behind API Controllers and what we mean by "Business Logic."

Concept	Analogy	What it means for our app
API Controller	A specialized chef or department in a company	A dedicated Python module (a `.py` file) that groups together related functions for a specific feature or area of the application. For example, all functions related to user dashboards.
Business Logic	The specific recipe and steps for cooking a dish	The actual "brains" of the application. It's the set of rules, calculations, and data processing steps needed to fulfill a request. It defines *what* the application does.

Concept	Analogy	What it means for our app
Modularity	Breaking down a big kitchen into stations	Organizing code into smaller, independent, and reusable pieces. Controllers help achieve this by separating specific features into their own files.

In our `MAD-2-Project-iitm` project, you'll find these API Controllers organized neatly in the `backend/controllers/` folder, often further categorized by `user`, `admin`, `exam`, or `jobs`.

Use Case: Fetching User Summary Data

Let's take the example of displaying a user's quiz summary on their profile page. The frontend needs information like total quizzes attempted, average score, and performance per subject. This is a perfect job for an API Controller!

The primary controller for this task is located at `backend/controllers/user/user_summary.py`. It contains the function `user_summary(user_id)`.

How `app.py` Calls the Controller

First, our main Flask server (`app.py`) needs to know about this "specialized chef" (the `user_summary` controller). It imports the function and defines a route where the frontend can ask for this information.

Here's a simplified look at how `app.py` might call the `user_summary` controller:

```
# File: backend/app.py (Simplified Excerpt)
from flask import request, jsonify
# Import our specialized chef!
from controllers.user.user_summary import user_summary
# (Assume check_token function is also imported)

@app.route('/api/user_summary', methods=['GET'])
def get_user_summary_api():
    # 1. Get the user's digital pass (token) from the request
    token = request.args.get('token')

    # 2. Verify the token using Authentication logic (from Chapter 4)
    payload, error_response, status_code = check_token(token)
    if payload is None:
        return jsonify({"message": error_response}), status_code

    user_id = payload['user_id'] # Get the user's ID

    # 3. Call our API Controller function to do the heavy lifting!
    summary_data = user_summary(user_id)

    # 4. Send back the processed data to the frontend
    if summary_data:
        return jsonify(summary_data), 200
    return jsonify({"message": "Summary not available"}), 500
```

Explanation:

- `from controllers.user.user_summary import user_summary`: This line is like the dispatcher (app.py) knowing the name and location of a specific chef (`user_summary`).
- `@app.route('/api/user_summary', methods=['GET'])`: This tells Flask that when the frontend sends a `GET` request to `/api/user_summary`, it should run the `get_user_summary_api` function.
- `user_id = payload['user_id']`: After validating the user's token (as learned in Chapter 4: Authentication & Session Management), `app.py` extracts the `user_id`. This is crucial because the controller needs to know *which user's* summary to calculate.
- `summary_data = user_summary(user_id)`: This is the moment `app.py` hands off the request to our API Controller. It says, "Hey `user_summary` chef, here's the `user_id`, please calculate everything for this user!"
- `return jsonify(summary_data), 200`: The `user_summary` controller returns the prepared summary data, and `app.py` formats it as JSON and sends it back to the frontend.

The Controller in Action: `user_summary.py`

Now let's look at the "chef" itself, the `user_summary` controller. This is where the actual **business logic** resides.

```
# File: backend/controllers/user/user_summary.py (Simplified Excerpt)
from models.model import User, UserQuizAttempt, Quiz # Import blueprints!
from database import db # Our special database gateway

def user_summary(user_id):
    # 1. Start with an empty "summary report"
    summary = {
        "total_quiz_attempts": 0,
        "avg_percentage": 0,
        # ... many more fields for other stats ...
    }

    # 2. Get the user from the database using their ID (from Chapter 1)
    user = User.query.filter_by(uuid=user_id).first()
    if not user:
        return None # User not found

    # 3. Fetch ALL quiz attempts for this user from the database
    all_attempts = UserQuizAttempt.query.filter_by(user_uuid=user_id).all()

    # 4. Do a simple calculation: how many attempts?
    summary["total_quiz_attempts"] = len(all_attempts)

    total_percentage_sum = 0
    # 5. Loop through each attempt to calculate scores
    for attempt in all_attempts:
        quiz = Quiz.query.get(attempt.quiz_uuid) # Get quiz details
        if quiz and quiz.max_score > 0:
            percentage = (attempt.score * 100) / quiz.max_score
            total_percentage_sum += percentage
```

```
    # 6. Calculate the overall average percentage
    if summary['total_quiz_attempts'] > 0:
        summary['avg_percentage'] = round(total_percentage_sum /
summary['total_quiz_attempts'], 2)

    # ... The controller would perform many more calculations here ...
    # ... like month-wise attempts, subject-wise scores, etc. ...

    return summary # Return the complete summary report
```

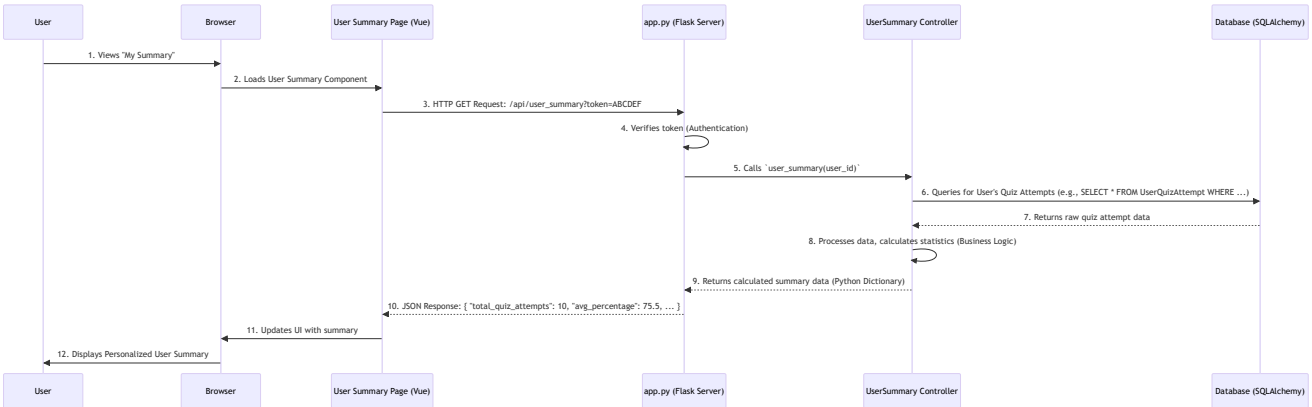
Explanation:

- `from models.model import ...`: The controller imports the necessary Database Models (`User`, `UserQuizAttempt`, `Quiz`, etc.) because it needs to fetch data from the database.
- `user = User.query.filter_by(uuid=user_id).first()`: This is an example of querying the database. The controller uses SQLAlchemy (from Chapter 1: Database Models) to find the specific user record.
- `all_attempts = UserQuizAttempt.query.filter_by(user_uuid=user_id).all()`: This fetches *all* the records from the `UserQuizAttempt` table that belong to our `user_id`.
- `summary["total_quiz_attempts"] = len(all_attempts)`: Here, the controller performs a simple calculation – counting the number of attempts.
- `for attempt in all_attempts: ...`: The controller loops through the data, performs more complex calculations (like percentages), and gathers information needed for the summary.
- `return summary`: Finally, the controller returns a well-structured `summary` dictionary, containing all the calculated results. This dictionary is then sent back to `app.py`.

Notice that this controller doesn't care *how* the request arrived (that's `app.py`'s job), and it doesn't care *how* the data is presented to the user (that's the frontend's job, from Chapter 2: Frontend Views). Its sole focus is on fetching, processing, and calculating the raw data according to the application's rules. That's the **business logic**!

What Happens Under the Hood?

Let's visualize the flow from when a user wants their summary to when they see it on screen.



1. **User Request**: The user navigates to their "My Summary" page.
2. **Frontend Component**: The relevant Frontend View (Vue Component) loads in the browser.

3. **HTTP Request**: The Vue Component sends an HTTP `GET` request to our `FlaskServer` (the `app.py`) asking for the user's summary data, including the user's JWT token for authentication.
4. **Flask Server Receives**: `app.py` receives the request. It first uses its Authentication & Session Management logic to verify the user's token.
5. **Dispatch to Controller**: If the token is valid, `app.py` then calls the `user_summary(user_id)` function from the `UserSummary Controller`.
6. **Controller Queries Database**: The `UserSummary Controller` interacts with the `db` object and the Database Models (SQLAlchemy ORM) to fetch all the necessary raw data (like all quiz attempts and quiz details) from the database.
7. **Database Responds**: The database returns the raw data to the controller.
8. **Business Logic Execution**: The `UserSummary Controller` then applies all its "business logic": it loops through the raw data, performs calculations (counts, averages, percentages), and organizes it into a structured dictionary (our `summary`).
9. **Controller Returns Data**: The controller sends this well-structured, processed `summary` dictionary back to `app.py`.
10. **Flask Sends Response**: `app.py` takes the dictionary, converts it into a standard **JSON** format, and sends it back as an HTTP response to the frontend.
11. **Frontend Updates**: The Vue Component receives the JSON data and updates its internal variables, which in turn causes the user interface to display the personalized summary.

This separation of concerns—`app.py` handles the routing and authentication, and `API Controllers` handle the specific business logic and database interactions—makes our backend clean, organized, and easier to manage.

Other Examples of API Controllers

Our project uses many different API Controllers, each handling a specific set of related tasks. Here are a few examples from the `backend/controllers/` directory:

Controller File	Purpose (What Business Logic it handles)	Example Function Calls (Simplified)
`user/user_search.py`	Processes user search queries for quiz attempts based on different criteria.	`user_search_result(user_id, parameter, query)`
`exam/store_answer.py`	Calculates quiz scores, stores user answers and quiz attempt results.	`store_ans(user_id, quiz_id, attempt_number, answers_data)`
`admin/create.py`	Handles all logic for creating new levels, subjects, chapters, quizzes, questions.	`cre_level(id, name, description)` or `cre_quiz(uuid, title, ...)`
`admin/delete.py`	Contains logic for deleting content (levels, subjects, chapters, quizzes, questions).	`delete_chapter(chapter_id)`
`admin/user_control.py`	Manages administrative actions like finding, blocking, or unblocking users.	`block_user(user_id)` or `find_usr(search_pattern)`

Controller File	Purpose (What Business Logic it handles)	Example Function Calls (Simplified)
`admin/summary.py`	Gathers comprehensive statistics for the admin dashboard.	`get_admin_summary()`

As you can see, each controller is a dedicated "department" focused on its own set of business rules and operations, making the overall backend much more manageable.

Summary

In this chapter, we learned that:

- **API Controllers** are specialized Python modules in our backend that contain the **business logic** for specific features.
- They act like "specialized chefs" or "departments," taking over from `app.py` to perform detailed data processing and calculations.
- They extensively interact with our Database Models (SQLAlchemy ORM) to fetch and manipulate data.
- They return the processed data back to the Backend API Server (Flask `app.py`), which then sends it to the frontend.
- This structure promotes **modularity** and keeps our code organized and efficient.

Understanding API Controllers is vital to seeing how the application's "brain" works. In our next and final chapter, we'll explore **Asynchronous Task Queue (Celery/Redis)**, learning how we handle long-running or scheduled tasks in the background without slowing down the user experience.

Next Chapter: Asynchronous Task Queue (Celery/Redis)

Chapter 7: Asynchronous Task Queue (Celery/Redis)

Welcome to our final chapter! In our previous chapter: API Controllers (Business Logic), we learned how our backend uses specialized "chefs" (API Controllers) to handle complex calculations and interactions with the database. We saw how a controller efficiently gathers all the data for a user's quiz summary.

But what if a "dish" (task) takes a very, very long time to prepare? Imagine our application needs to:

- Send personalized emails to thousands of users.
- Generate a complex, data-heavy monthly performance report for every user.
- Check for new quizzes and notify relevant users every day at a specific time.

If our main Flask server (`app.py`) tried to do these long tasks immediately when a user requests them, or at a scheduled time, it would get stuck! The user would see a slow, unresponsive website, or the scheduled task might block other important operations. This is where the **Asynchronous Task Queue** comes to our rescue!

What's the Big Idea? (Motivation)

Think of our application as a popular coffee shop.

- The **barista** (our main Flask server, `app.py`) is great at quickly taking orders (user requests) and serving simple coffees (fast API responses).
- Sometimes, a customer orders a very complicated, custom-blended, slow-drip coffee (a long-running task like generating a big report or sending many emails).
- If the barista stops everything to make *just that one* complex coffee, all other customers (users) behind them will get frustrated waiting!

This is where a **specialized "back-room" team** comes in.

Instead of the barista making the complex coffee:

1. The barista quickly takes the order and gives the customer a **pager** (an immediate response, saying "Your complex order is being prepared!").
2. The barista then sends the complex order to the **back-room team's "to-do list"** (our **Task Queue**).
3. The **back-room chefs** (our **Celery Workers**) then pick up items from this "to-do list" and prepare them quietly, without bothering the barista or other customers.
4. Meanwhile, the barista is free to keep serving simple coffees to other customers, keeping the main shop fast and happy.
5. There's also a **scheduler** (our **Celery Beat**) in the back room. This person is like an alarm clock, making sure certain complex coffees (like "monthly special blends") are automatically started at specific times, even if no one explicitly ordered them right then.

This "back-room" setup, using **Celery** for the specialized team and **Redis** for the "to-do list" (and also the pagers), ensures our `MAD-2-Project-iitm` application remains super fast and responsive for users, even when there are big jobs to do!

Our goal in this chapter is to understand how Celery and Redis work together to handle these background tasks, allowing our application to perform long-running operations efficiently without blocking the main website.

Key Concepts Explained

Let's break down the core ideas behind Asynchronous Task Queues.

Concept	Analogy	What it means for our app (`MAD-2-Project-iitm`)
Synchronous Task	Barista makes coffee immediately	When the main server (Flask `app.py`) does a task right away, waiting for it to finish before responding to the user. Good for quick tasks (e.g., login, fetching small data).
Asynchronous Task	Barista gives pager, back-room makes coffee	When the main server quickly sends a task to a background system and immediately responds to the user. The task runs independently. Ideal for long-running operations (e.g., sending emails).
Task Queue	The "to-do list" for the back-room team	A temporary storage area where tasks are placed, waiting to be processed by workers.
Celery	The "specialized back-room team" manager	A powerful Python library for managing distributed task queues. It defines how tasks are created, sent, and executed.
Celery Worker	The "back-room chef" doing the cooking	A separate process that constantly watches the Task Queue, picks up tasks, and executes them in the background.
Celery Beat	The "alarm clock/scheduler"	A separate process that reads a schedule and periodically adds tasks to the queue, ensuring they run at specific times (e.g., daily new quiz notifications, monthly reports).
Redis	The "post office" or "message board"	A super-fast, in-memory database used by Celery as both a **message broker** (where tasks are initially placed) and a **result backend** (where results of tasks can be stored). It's the communication hub.

Setting Up Celery and Redis in `app.py` and `config.py`

Before our application can use Celery and Redis, we need to tell them how to work together. This setup happens in `backend/app.py` and `backend/config.py`.

1. Flask App Initializes Celery

In `app.py`, after setting up our main Flask application, we also initialize Celery and tell it about our Redis connection.

```
# File: backend/app.py (Excerpt)
from celery import Celery # Import Celery

# ... (other imports and Flask app setup) ...
```

```
# ============================================================
# === Load Celery with config ===
# ============================================================
celery = Celery(app.name) # Create a Celery instance
celery.conf.update(
    broker_url=app.config['CELERY_BROKER_URL'],       # Where tasks are sent
(Redis)
    result_backend=app.config['CELERY_RESULT_BACKEND'], # Where task results are
stored (Redis)
    imports=app.config['IMPORTS'],                     # Where our task functions
are defined
    beat_schedule=app.config['BEAT_SCHEDULE']          # Schedule for recurring
tasks
)
```

Explanation:

- `celery = Celery(app.name)`: This creates a Celery application instance, linking it to our Flask app.
- `celery.conf.update(...)`: This configures Celery using settings from `app.config`.
  - `broker_url`: This is the address of our Redis server (`redis://localhost:6379/1`). Redis acts as the **message broker**, the "to-do list" where tasks are put.
  - `result_backend`: This is also Redis. After a Celery task finishes, if it has a result, it can store it here.
  - `imports`: This tells Celery where to find our actual task functions (e.g., in `controllers.jobs.celery_tasks`).
  - `beat_schedule`: This links to our schedule for periodic tasks, which Celery Beat will use.

2. Configuration for Celery and Redis in `config.py`

Our `backend/config.py` file holds all the important settings, including the URLs for Redis and the schedule for `Celery Beat`.

```
# File: backend/config.py (Excerpt)
import os
from dotenv import load_dotenv
from celery.schedules import crontab # For scheduling tasks

load_dotenv()

class Config:
    # ... (other configurations like SQLALCHEMY_DATABASE_URI, SECRET_KEY) ...

    # Redis for caching (Chapter 5)
    CACHE_TYPE = 'RedisCache'
    CACHE_REDIS_URL = os.environ['REDIS_CACHE_URL'] # Typically
redis://localhost:6379/0

    # Redis for Celery (Different database from cache for clarity)
    CELERY_BROKER_URL = os.environ.get('CELERY_BROKER_URL') # Typically
redis://localhost:6379/1
```

```
    CELERY_RESULT_BACKEND = os.environ.get('CELERY_RESULT_BACKEND_URL') # Same as
broker for simplicity

    IMPORTS = ('controllers.jobs.celery_tasks',) # Path to our task definitions

    BEAT_SCHEDULE = {
        'send-email-of-any-new-quiz': { # Name of this scheduled task
            'task':
'controllers.jobs.celery_tasks.check_new_created_quiz_24_hrs_ago',
            'schedule': crontab(hour=18, minute=0), # Run daily at 6 PM UTC
        },
        'send-email-monthly-report':{ # Another scheduled task
            "task": 'controllers.jobs.celery_tasks.send_email_report',
            "schedule": crontab(day_of_month=1, hour=1, minute=0), # Run on 1st of
every month at 1 AM UTC
        }
    }

    # ... (Email server info for sending reports/notifications) ...
    MAIL_SERVER = os.environ.get('MAIL_SERVER')
    MAIL_PORT = int(os.environ.get('MAIL_PORT', 465))
    MAIL_USERNAME = os.environ.get('MAIL_USERNAME')
    MAIL_PASSWORD = os.environ.get('MAIL_PASSWORD')
```

Explanation:

- `CELERY_BROKER_URL` and `CELERY_RESULT_BACKEND`: These tell Celery the address of our Redis server. Notice that Redis can be used for both caching (from Chapter 5: Backend API Server) and Celery's messaging, often on different "databases" (indicated by `/0` and `/1` in the URL) to keep things separate.
- `IMPORTS`: This is a tuple (a list of items) that points to the Python module (`controllers.jobs.celery_tasks`) where our background tasks are defined.
- `BEAT_SCHEDULE`: This is a dictionary that defines tasks which should run automatically on a schedule.
  - `'send-email-of-any-new-quiz'`: This is a unique name for our task.
  - `'task'`: This specifies the full path to the Python function that Celery should execute.
  - `'schedule'`: This uses `crontab` (a common scheduling syntax) to define *when* the task should run (e.g., `hour=18, minute=0` means 6 PM UTC daily). There is also an option to define a simple `schedule` in seconds for testing (e.g., `15.0`).

3. Starting Celery Processes

The `app.py` file also contains the logic to start the Celery Worker and Celery Beat processes when our main Flask application is launched.

```
# File: backend/app.py (Excerpt)
import atexit
import subprocess # To run external commands

# ... (other imports and app setup) ...

processes = [] # To keep track of our background processes
```

```
def start_background_services():
    """A helper function to start the Celery services."""
    print("--- Starting Background Services ---")

    # Command to start the Celery Worker (the chef)
    worker_cmd = ["celery", "-A", "app.celery", "worker", "--pool=solo", "--
loglevel=info"]
    worker_p = subprocess.Popen(worker_cmd) # Start the worker
    processes.append(worker_p)
    print(f"Celery Worker started with PID: {worker_p.pid}")

    # Command to start the Celery Beat (the scheduler/alarm clock)
    beat_cmd = ["celery", "-A", "app.celery", "beat", "--loglevel=info"]
    beat_p = subprocess.Popen(beat_cmd) # Start the beat
    processes.append(beat_p)
    print(f"Celery Beat started with PID: {beat_p.pid}")

def stop_background_services():
    """A helper function to stop services when CTRL+C is pressed."""
    print("\n--- Stopping all background services... ---")
    for p in processes:
        p.terminate() # Gracefully ask processes to stop
    print("--- Shutdown complete. ---")

if __name__ == '__main__':
    # This check ensures Celery starts only once, even with Flask's reloader
    if os.environ.get("WERKZEUG_RUN_MAIN") != "true":
        with app.app_context():
            db.create_all() # Create database tables (from Chapter 1)
        start_background_services() # Start our Celery worker and beat
        atexit.register(stop_background_services) # Ensure cleanup on exit

    app.run(debug=True, port=5000) # Start the Flask web server
```

Explanation:

- `subprocess.Popen(...)`: This Python function runs a command in a new background process. This is how we launch the `celery worker` and `celery beat` commands.
- `celery -A app.celery worker`: This command starts a Celery Worker. `-A app.celery` tells Celery to load its configuration from the `celery` object we created in `app.py`. `--pool=solo` is important for Windows users.
- `celery -A app.celery beat`: This command starts the Celery Beat scheduler.
- `atexit.register(stop_background_services)`: This ensures that when you press `Ctrl+C` to stop the Flask server, our `stop_background_services` function runs, which gracefully shuts down the Celery worker and beat processes.

Defining Asynchronous Tasks (`celery_tasks.py`)

Our actual background tasks live in `backend/controllers/jobs/celery_tasks.py`. Any function that you want Celery to run asynchronously must be decorated with `@celery.task`.

Example 1: Sending New Quiz Notifications (Scheduled Task)

This task is defined in `BEAT_SCHEDULE` and will be triggered by Celery Beat daily.

```
# File: backend/controllers/jobs/celery_tasks.py (Simplified Excerpt)
from app import celery, app # Import our celery instance and Flask app
from datetime import datetime, timedelta
from models.model import Quiz, User, UserQuizAttempt # Our DB Models
import smtplib # For sending emails
from flask import render_template # To render email templates

@celery.task
def check_new_created_quiz_24_hrs_ago():
    """Scheduled task to find new quizzes and notify relevant users."""
    with app.app_context(): # Essential for accessing Flask app context (DB,
configs)
        print("\n--- SCHEDULED TASK: Checking for new quizzes ---")
        twenty_four_hours_ago = datetime.utcnow() - timedelta(hours=24)
        latest_quizzes = Quiz.query.filter(Quiz.created_at >=
twenty_four_hours_ago).all()

        if not latest_quizzes:
            print("No new quizzes found in the last 24 hours.")
            return

        print(f"Found {len(latest_quizzes)} new quizzes. Processing...")
        for quiz in latest_quizzes:
            # Find users who have bookmarked chapters related to this quiz
            search_term = f"%{quiz.chapter_uuid}%"
            relevant_users =
User.query.filter(User.user_selected_chapter.like(search_term)).all()

            for user in relevant_users:
                # Check if user has already attempted this quiz
                attempt = UserQuizAttempt.query.filter_by(user_uuid=user.uuid,
quiz_uuid=quiz.uuid).first()
                if not attempt:
                    print(f"Sending new quiz notification to {user.username} for
'{quiz.title}'")
                    # Send this task to Celery to process in the background!
                    send_email_to_user.delay(user.uuid, quiz.uuid)
                else:
                    print(f"User {user.username} has already attempted quiz
'{quiz.title}'.")
        print("New quiz check finished.")

@celery.task
def send_email_to_user(user_id, quiz_id):
    """Sends a single email to a single user about a new quiz."""
    with app.app_context(): # Also needs app context for DB access and mail config
        print(f"\n--- ASYNC TASK: Sending email to user {user_id} for quiz
{quiz_id} ---")
```

```
        try:
            user = User.query.get(user_id)
            quiz = Quiz.query.get(quiz_id)
            # ... (Logic to render email template and send email using smtplib)
...
            print(f"SUCCESS: Email sent to {user.email}")
            return "Email sent successfully!"
        except Exception as e:
            print(f"ERROR: Could not send email. Reason: {e}")
            return f"Email failed: {str(e)}"
```

Explanation:

- `@celery.task`: This special decorator is crucial! It tells Celery that the function immediately below it (`check_new_created_quiz_24_hrs_ago` or `send_email_to_user`) is a Celery task and can be run in the background.
- `with app.app_context()`: When Celery runs a task, it's a separate process. It doesn't automatically know about our Flask `app` or `db` object. This line manually sets up the Flask application context, allowing the task to access our database models (Chapter 1), configurations (Chapter 5), and other Flask-specific resources.
- `send_email_to_user.delay(user.uuid, quiz.uuid)`: This is how we tell Celery to run a task *asynchronously*. Instead of calling `send_email_to_user(...)` directly (which would run it immediately and block), we use `.delay()`. This sends the task to the Redis "to-do list," and the main process moves on without waiting.

Example 2: Sending Monthly Performance Reports (Scheduled Task)

This is another scheduled task that runs periodically to send detailed reports.

```
# File: backend/controllers/jobs/celery_tasks.py (Simplified Excerpt)
# ... (imports like from app import celery, app, etc.) ...

@celery.task
def send_email_report():
    """Scheduled task to send a monthly performance report to all active users."""
    with app.app_context():
        print("\n--- SCHEDULED TASK: Starting monthly report job ---")
        # Get active users
        all_users = User.query.filter_by(is_active=True).all()

        if not all_users:
            print("No active users found. Stopping job.")
            return

        for user in all_users:
            try:
                print(f"Processing report for user: {user.username}")
                # ... (Complex logic to fetch user's quiz attempts, calculate
scores,
                #      find strongest/weakest subjects, prepare data for email)
```

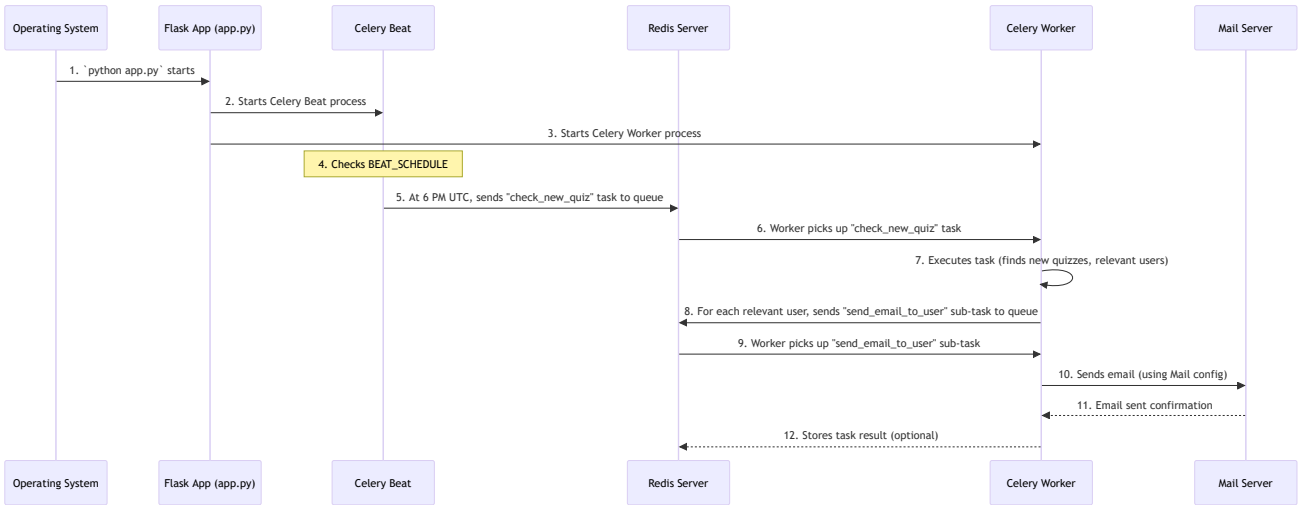
```
...

                # Render email template and send email
                # This part is similar to send_email_to_user but with more data
                # ... (smtplib logic) ...
                print(f"SUCCESS: Report sent to {user.email}")

            except Exception as e:
                print(f"ERROR: Failed to send report to {user.username}. Reason:
{e}")

        print("Monthly report job finished.")
        return "Monthly reports process completed."
```

Explanation:

- Like the previous task, `send_email_report` is also decorated with `@celery.task` and runs within `app.app_context()`.
- This task demonstrates a long-running operation that processes data for multiple users and sends individual emails. Doing this directly within a Flask API endpoint would be too slow and would block the user's browser. By making it a Celery task, the process runs silently in the background, consuming resources without affecting user-facing performance.

What Happens Under the Hood?

Let's visualize how a scheduled task, like checking for new quizzes, is handled by Celery and Redis.



1. **Application Startup**: When you run `python app.py`, the `if __name__ == '__main__':` block executes. It initializes the database and then uses `subprocess.Popen` to start two crucial background processes: `Celery Beat` and `Celery Worker`.
2. **Celery Beat Schedules**: `Celery Beat` starts running. It continuously looks at the `BEAT_SCHEDULE` defined in `config.py`. When the scheduled time arrives (e.g., 6 PM UTC for `check_new_created_quiz_24_hrs_ago`), Celery Beat creates a message for that task.
3. **Task to Redis (Broker)**: Celery Beat sends this message (the task) to the `Redis Server`, which acts as the **message broker**. Redis puts this task onto a queue, waiting to be picked up.

4. **Celery Worker Picks Up Task**: Our `Celery Worker` is constantly listening to this queue in Redis. As soon as a task appears, a worker picks it up.
5. **Worker Executes Task**: The Celery Worker executes the Python function associated with the task (e.g., `check_new_created_quiz_24_hrs_ago`). This function runs in a separate process, so it doesn't slow down the main Flask server.
    - Inside `check_new_created_quiz_24_hrs_ago`, it performs database queries (using our Database Models) to find new quizzes and relevant users.
    - For each relevant user, it then uses `send_email_to_user.delay()` to send *another* task to Celery. This creates even more background tasks!
6. **Sub-tasks to Redis**: Each `send_email_to_user.delay()` call sends a new message (a sub-task) to the Redis queue.
7. **Worker Picks Up Sub-tasks**: Other Celery Workers (or the same one, if it finishes quickly) pick up these individual email-sending tasks.
8. **Email Sending**: Each `send_email_to_user` task connects to the mail server (using settings from `config.py`) and sends an actual email. This is an I/O (Input/Output) bound operation that can take time, but it's happening completely in the background.
9. **Task Completion**: Once an email is sent, the task is marked as complete. Any results (optional) can be stored back in Redis (the `result_backend`).

This entire process allows the `MAD-2-Project-iitm` application to handle heavy loads and scheduled operations smoothly, without impacting the user's immediate experience.

Summary

In this final chapter, we discovered the power of **Asynchronous Task Queues (Celery/Redis)**:

- **Asynchronous tasks** allow our application to perform long-running or scheduled operations in the background, keeping the main website fast and responsive for users.
- **Celery** is our powerful Python tool for managing these tasks, acting as the "specialized back-room team."
- **Redis** serves as the vital communication hub, functioning as both the **message broker** (where tasks are queued) and the **result backend** (where task results can be stored).
- **Celery Workers** are the dedicated processes that pick up and execute tasks from the queue.
- **Celery Beat** is the scheduler that automatically dispatches tasks to the queue at predefined intervals (e.g., daily new quiz notifications, monthly reports).
- We define tasks using the `@celery.task` decorator and ensure they run within the Flask `app.app_context()` when they need to interact with our application's resources (like the database).
- We send tasks to the queue using the `.delay()` method, which immediately returns control to the main application.

This concludes our comprehensive tutorial for the `MAD-2-Project-iitm`. You've learned about every major component, from how data is structured in the database to how it's presented to the user, how security is maintained, and how complex operations are handled efficiently in the background.

Congratulations on completing this journey! You now have a solid understanding of how all the pieces of this modern web application fit together.

✨ ✨ ✨ ✨ The End ✨ ✨ ✨ ✨

Created with ♥ and passion by Daiwik Rankawat.